Final Year Design Project

Construction Material

(Wholesale & Retail Marketplace Platform)



By

Muhammad Nosherwan	**076926**
Hafiz Abdul Rehman	**077011**
Talha Tariq	**077019**
Project ID:	**BSIT-22-G2**

Under the Supervision of

Prof. Usman Ahmed

Bachelor of Science in Information Technology (2022-2026)

GOVT. GRADUATE ISLAMIA COLLEGE, GUJRANWALA

A project presented to
University of the Punjab, Lahore

In partial fulfillment of the
requirement for the degree of

Bachelor of Science in Information Technology (2022-2026)

By

Muhammad Nosherwan	**076926**
Hafiz Abdul Rehman	**077011**
Talha Tariq	**077019**

FACULTY OF COMPUTING & INFORMATION TECHNOLOGY (FCIT),

GOVT Islamia Graduate COLLEGE, GUJRANWALA

DECLARATION

We hereby declare that this software, neither whole nor as a part, has been copied out from any source. It is further declared that we have developed this web-app and accompanied report entirely based on our personal efforts. If any part of this project is proved to be copied out from any source or found to be reproduction of some other, we will stand by the consequences. No portion of the work presented has been submitted of any application for any other degree or qualification of this or any other university or institute of learning.

Muhammad Nosherwan

Signature: _____

Hafiz Abdul Rehman

Signature: _____

Talha Tariq

Signature: _____

CERTIFICATE OF APPROVAL

It is to certify that the final year design project (FYDP) of BSIT "Construction Material (Wholesale & Retail)" was developed by **Muhammad Nosherwan[076926], Hafiz Abdul Rehman[077011] and Talha Tariq[077019]** under the supervision of "Prof. Usman Ahmed" in my opinion; it is fully adequate, in scope and quality for the degree of Bachelor of Science in IT.

FYDP Supervisor: Prof. Usman Ahmed

Signature: ----------

Signatures (Faculty Advisory Committee (FAC)

	FAC1	FAC2	FAC3
Name	**Prof. Usman Ahmed**	**Prof. Muhammad Asif**	**Prof. Imran Shafique**
Signature			

Head of FYDP Coordination Office:

Signature: ----------

Chairperson, Department of [BSIT] Dated: _____

Signature: ----------

Executive Summary

The *Construction Material* is a web-based marketplace platform designed to connect suppliers, wholesalers, retailers and customers for buying and selling construction materials in Pakistan. The system provides a centralized platform where users can browse products, compare prices, place orders and manage inventory online.

The main purpose of this project is to solve the problem of inefficiency and lack of transparency in Pakistan's construction material market. Currently, buyers have to contact multiple sellers manually to check prices and availability. This system overcomes these issues by providing a digital, transparent and efficient marketplace.

The platform supports role-based dashboards for Admin, Supplier, Wholesaler, Retailer and Customer. Each role has specific permissions and features. An admin verifies sellers before their products are listed. Sellers manage their product listings and inventory. Buyers can place orders, request bulk quotes and track order status.

The system is developed using React.js, Node.js, Express.js and PostgreSQL, ensuring scalability, security and efficient data handling. It also includes features like Stripe payment integration, JWT authentication, Nodemailer email notifications, PDF invoice generation and image uploading via Multer.

In conclusion, the *Construction Material* platform is a user-friendly and efficient solution that helps buyers and sellers in the construction industry transact with transparency, speed and confidence by providing a reliable, centralized and organized digital marketplace.

Acknowledgement

We express our gratitude to our **Almighty Allah,** and after that, we are thankful to our **Parents**, who facilitated us to get higher education, and we became capable of creating something productive. We're also thankful to our project advisor **Prof. Usman Ahmed** and the department faculty for their guidance and support throughout this project.

Muhammad Nosherwan

Signature: _____

Hafiz Abdul Rehman

Signature: _____

Talha Tariq

Signature: _____

Abbreviations

All the abbreviations and acronyms used in our FYP along with their respective brief description.

Abbreviations	Full Form	Description
API	Application Programming Interface	A set of protocols that allow different software systems to communicate.
CSS	Cascading Style Sheet	A style sheet language used for describing the presentation of a web page.
DB	Database	A structured collection of data stored efficiently.
HTML	Hyper Text Markup Language	The standard markup language for creating web pages.
JS	JavaScript	A programming language used for client-side scripting on web.
ORM	Object Relational Mapping	A tool that lets you interact with a database using objects instead of raw SQL queries.
Node.js	Node JavaScript	A runtime environment for executing JavaScript on the server side.
React	React.js	A JavaScript library for building fast and dynamic user interfaces.
UI	User Interface	The part of a system with which users interact.
JWT	JSON Web Token	A token-based standard used for secure user authentication and session management.
CORS	Cross-Origin Resource Sharing	A security mechanism that enables secure communication between the frontend and backend across different origins.
MVC	Model View Controller	An architectural pattern used to separate application logic into Model, View and Controller layers.
REST	Representational State Transfer	An architectural style for designing APIs using standard HTTP methods like GET, POST, PUT and DELETE.
SMTP	Simple Mail Transfer Protocol	The protocol is used by Nodemailer to send automated email notifications in our system.
PostgreSQL	PostgreSQL	A relational database system is used to store application data.
Bcrypt	Bcrypt Password Hashing	A library used to securely hash and store user passwords in the database.
Stripe	Stripe Payment Gateway	A PCI-compliant third-party payment gateway used to process secure online transactions.
Multer	Multipart Upload Middleware	A Node.js middleware used to handle product image uploads from the supplier dashboard.
Axios	Axios HTTP Client	A JavaScript library is used in the React.js frontend to send and receive HTTP requests to the backend API.

Table of Contents

Chapter 1: 13

Introduction 13

1. Introduction 14

1.1 Problem Statement 14

1.2 Problem Solution 14

1.3 Objectives of proposed system 15

1.4 Scope 15

1.5 System Components 15

1.6 Related System Analysis / Literature Review 18

1.7 Vision statement 18

1.8 System limitations and constraints 18

1.9 Tools and Technologies 19

1.10 Project deliverables 20

1.11 Project planning 21

1.12 Summary 22

Chapter 2: 23

Analysis 23

2. Analysis 24

2.1 User Classes and Characteristics 24

2.2 Requirement Identifying Techniques 24

2.3 Functional Requirements 25

2.4 Non-Functional Requirements 31

2.5 Use Case Analysis 32

2.6 External Interface Requirements 47

Chapter 3: 50

System Design 50

3. System Design 51

3.1 Design Considerations 51

3.2 Design Models 52

3.3 Architectural Design 60
3.4 Data Design 62
3.5 User Interface Design 67
3.5.1 Screen Images 70
3.6 Design Decisions 74
3.7 Summary 75
Chapter 4: 76
Implementation 76
4. Implementation 77
4.1 Algorithms 77
4.2 External APIs / SDKs 84
4.3 Code Repository 86
4.4 Summary 86
Chapter 5: 87
System Testing 87
5. Introduction 88
5.1 Unit Testing (UT) 88
5.2 Functional Testing (FT) 93
5.3 Integration Testing (IT) 99
5.4 Summary 105
Chapter 6: 106
System Conversion 106
6 Introduction 107
6.1 Conversion Method 107
6.2 Deployment 108
6.3 Post Deployment Testing 111
6.4 Challenges and Solutions 112
6.5 Summary 113
Chapter 7: 114
Conclusion 114

7.1 Evaluation 115
7.2 Requirements Traceability Matrix 116
7.3 Conclusion 117
7.4 Future Work 119
References 119
Appendix A: 121
Use Case Description 121
UC-1: User Registration 121
UC-2: User Login 122
UC-3: Seller Verification by Admin 122
UC-4: Manage Product Listings 123
UC-5: Browse and Search Products 124
UC-6: Place Order 125
UC-7: Make Payment and Download Invoice 126
UC-8: Submit Bulk Order Request for Bidding 127
UC-9: Submit and Manage Bids 128
UC-10: Track Order Status 129
UC-11: Admin User Management 130
UC-12: Admin Product Management 131
UC-13: Admin Order Management 131
UC-14: Generate System Reports 132
Appendix B: Coding Standards 133
General Coding Standards & Guidelines 133
Appendix C: Prototypes 136

List of Figures

Figure 1-1: Gantt Chart 21
Figure 2-1: Use Case Diagram 46

Figure 3 - 1: Class Diagram 53
Figure 3 - 2: Sequence Diagram 1 54
Figure 3 - 3: Sequence Diagram 2 55
Figure 3 - 4: Sequence Diagram 3 56
Figure 3 - 5: Sequence Diagram 4 57
Figure 3 - 6: Sequence Diagram 5 58
Figure 3 - 7: State Transition Diagram 59
Figure 3 - 8: UML Component Diagram 60
Figure 3 - 9: MVC Model 61
Figure 3 - 10: ER Diagram 69

List of Tables

Table 1 - 1: Tools & Technologies 19

Table 2 - 1: User classes & Characterization 24
Table 2 - 2: Functional Requirement 1 25
Table 2 - 3: Functional Requirement 2 26
Table 2 - 4: Functional Requirement 3 26
Table 2 - 5: Functional Requirement 4 27
Table 2 - 6: Functional Requirement 5 27
Table 2 - 7: Functional Requirement 6 28
Table 2 - 8: Functional Requirement 7 28
Table 2 - 9: Functional Requirement 8 29
Table 2 - 10: Functional Requirement 9 30
Table 2 - 11: Functional Requirement 10 30
Table 2 - 12: Non-Functional Requirements 31
Table 2 - 13: Use Case 1 32
Table 2 - 14: Use Case 2 33
Table 2 - 15: Use Case 3 34
Table 2 - 16: Use Case 4 35
Table 2 - 17: Use Case 5 36
Table 2 - 18: Use Case 6 37
Table 2 - 19: Use Case 7 38
Table 2 - 20: Use Case 8 39
Table 2 - 21: Use Case 9 40
Table 2 - 22: Use Case 10 41
Table 2 - 23: Use Case 11 42
Table 2 - 24: Use Case 12 43
Table 2 - 25: Use Case 13 43

Table 2 - 26: Use Case 14 44

Table 4 - 1: Algorithm 1 77
Table 4 - 2: Algorithm 2 78
Table 4 - 3: Algorithm 3 79
Table 4 - 4: Algorithm 4 80
Table 4 - 5: Algorithm 5 80
Table 4 - 6: Algorithm 6 81
Table 4 - 7: Algorithm 7 82
Table 4 - 8: Algorithm 8 83
Table 4 - 9: Algorithm 9 83

Table 5 - 1: Unit Test 1 88
Table 5 - 2: Unit Test Case 2 89
Table 5 - 3: Unit Test Case 3 90
Table 5 - 4: Unit Test Case 4 92
Table 5 - 5: Functional Test Case 1 93
Table 5 - 6: Functional Test Case 2 94
Table 5 - 7: Functional Test Case 3 95
Table 5 - 8: Functional Test Case 4 96
Table 5 - 9: Functional Test Case 5 97
Table 5 - 10: Integration Test Case 1 99
Table 5 - 11: Integration Test Case 2 100
Table 5 - 12: Integration Test Case 3 102
Table 5 - 13: Integration Test Case 4 103
Table 5 - 14: Integration Test Case 5 104

Table 6 - 1: Deployment Steps 108
Table 6 - 2: Data Conversion 109
Table 6 - 3: Training Instruction 1 110
Table 6 - 4: Training Instruction 2 110
Table 6 - 5: Post Deployment Testing Steps 111
Table 6 - 6: Challenges and Solutions 112

Table 7 - 1: Progress Table 115
Table 7 - 2: Traceability Matrix 116

Chapter 1:
Introduction

1. Introduction

This chapter introduces the construction material web app (wholesale & retail) for construction materials. It highlights the purpose of creating single digital online platform for buyers and sellers, reducing manual processes and improving transparency in this field.

1.1 Problem Statement

The construction material market in Pakistan currently faces issues like transparency, digital record-keeping & more. Buyers have to reach out to several sellers manually to check prices and availability of material, which causes problems and increased operational costs. There is no single marketplace in Pakistan that fully serves for both wholesale and retail requirements on a single platform like Zarea.pk that deals in both but primarily focuses on wholesale, Mughal steels only deal in steel and there are many other platforms that are doing the same things. Some deal in retail only and some only work in their related field. Our project aims to fill these gaps by creating a web-based marketplace that provides the facility to buy & sell construction material online, providing greater transparency & efficiency.

1.2 Problem Solution

There are some solutions to solve the problem:

- Our Platform connects buyers directly with verified sellers, removing the need to contact manually sellers.
- Role-based modules for Admin, Supplier, Wholesaler, Retailer and Customer allow each user to manage their work efficiently.
- A pricing engine supports both wholesale and retail pricing so buyers can compare prices on single platform.
- Order placement and tracking features allow users to place and monitor orders online.
- Stripe payment integration and digital invoice generation provide secure transactions.
- Bidding for bulk orders allows multiple verified sellers to compete, giving buyers better prices.

1.3 Objectives of proposed system

- Add supplier, wholesaler, retailer and customer modules.
- Include product listing and order tracking.
- Integrate secure payment and invoice generation.
- Provide bidding for bulk orders.

1.4 Scope

Our system will function as a web-based platform. It will support product listings, order placement, supply management and payment processing. Future updates may include logistics and delivery tracking services. This digital solution aims to enhance buying and selling construction materials in minimized operational delays and encouraging fair pricing.

1.5 System Components

Construction Material (Wholesale & Retail) platform is based on five core modules, where each module is responsible for a specific set of functionalities. These modules work together to deliver a complete construction material marketplace experience for all user roles.

1.5.1 Module 1: Authentication & User Management Module

This module manages all user-related operations including registration, login and profile management. It handles Admin, Supplier, Wholesaler, Retailer and Customer accounts under a single user entity with role-based access control.

Users: Admin, Supplier, Wholesaler, Retailer, Customer

Key Functionalities:

- User Registration and Login.
- Role-Based Access Control.
- JWT Token Generation and Validation.
- Password Encryption (Bcrypt.js).
- Profile View and Update.

1.5.2 Module 2: Product & Inventory Management Module

This module allows sellers to list construction materials and manage their stock. Buyers can browse and search products, compare prices across sellers and view detailed product information before placing an order.

Users: Supplier, Wholesaler, Retailer

Key Functionalities:

- Add, Edit and Delete Product Listings.
- Upload Product Images via Multer.
- Browse and Search Products with Filters.
- Stock Tracking and Inventory Update.
- Support Wholesale and Retail Pricing

1.5.3 Module 3: Order & Payment Module

This module handles the complete order lifecycle from placement to completion. It is integrated with Stripe for secure online payments and automatically generates a digital invoice when an order is paid successfully.

Users: Wholesaler, Retailer, Customer, Supplier

Key Functionalities:

- Place Order and Select Products.
- Order Status Tracking.
- Secure Online Payment via Stripe.
- Automatic Digital Invoice Generation.
- View Order History.

1.5.3 Module 4: Bidding Module

This module provides bidding for bulk orders. When a buyer places a bulk order request, verified sellers can submit their bid amounts. The buyer can then compare bids and select the best offer, which promotes fair and competitive pricing for large purchases.

Users: Verified Seller

Key Functionalities:

- Submit Bulk Order Request.
- Submit and Cancel Bid.
- View and Compare All Bids.
- Accept Best Bid and Proceed to Order.

1.5.5 Module 5: Notification & Admin Reporting Module

This module handles all email notifications sent through Nodemailer and provides the admin with a reporting dashboard to monitor platform activity, manage users and oversee all transactions.

Users: Admin, all roles

Key Functionalities:

- Email Notifications for Order Status, Bid Updates and Price Changes.
- Verify and Approve seller accounts.
- Manage All Users, Products and Orders.
- Generate System Reports and Analytics Dashboard.

1.6 Related System Analysis / Literature Review

There are number of platforms but they focus on particular things & some of them are not available in Pakistan like:

- Zarea.pk: It is Pakistani construction marketplace, provides both wholesale and retail but primarily focuses on wholesale services.
- Alibaba: It is a global B2B marketplace but it is not localized in Pakistan.
- India Mart: It's a Indian marketplace for industrial & construction goods but it lacks in bidding.
- Construction Baazar: It is also a Pakistan marketplace and works in both wholesale and retail.
- Home Point: It also is a Pakistani marketplace but mainly serves in retail.

1.7 Vision statement

Here is the vision statement for the Construction Material platform:

For Suppliers, Wholesalers, Retailers and Customers in the Pakistani construction industry who need a transparent, efficient and fraud-free way to buy and sell construction materials online the Construction Material (wholesale & Retail) Platform is a centralized web-based marketplace that simplifies construction material buying and selling, ensures verified suppliers, supports bulk bidding and reduces frauds, existing platforms like Zarea.pk, Construction Baazar and Home Point that address only partial needs our product a unified, role-based marketplace with supplier verification, competitive bidding, secure payments and inventory management for Pakistan.

1.8 System limitations and constraints

Limitations:

- Real-time delivery tracking is not a part of current scope.
- Online payments depend on third party gateways.
- Image uploads are limited by server and Multer file size restrictions.
- External APIs may fail or affect the systems flow.

Constraints:

- The system is web-based only. Mobile application is not included in the current phase.
- Technology Stack: The project is built using React.js, Node.js, Express.js and PostgreSQL with Sequelize.

1.9 Tools and Technologies

Table 1 - 1: Tools & Technologies

Tools and Technologies	Rationale
VS Code	Lightweight, extensible IDE.
React.js	For creating dynamic and component-based user interfaces.
HTML, CSS, Bootstrap	For responsive and structured web interface.
Redux Toolkit	Manages global state efficiently.
Node.js	Execute JavaScript code on server side and handles logic.
Express.js	Simplify backend routes, APIs and server handling.
PostgreSQL	Storing data in structured form.
Sequelize	Connect Node.js with MySQL and simplifies queries.
JWT	For secure login and user session handling.

Bcrypt.js	Encrypt passwords and improves security overall.
Stripe	For online payment integration.
Nodemailer	For sending emails of price and feed updates.
Multer	For uploading images.
Axios	For sending and receiving data between frontend and backend.
CORS	Enable secure communication between frontend and backend.

Work Division

Sr No.	Name	Roll No.	Role Assignment & Work Division
1.	Muhammad Nosherwan	076926	Backend + Documentation
2.	Hafiz Abdul Rehman	077011	Backend + Documentation
3.	Talha Tariq	077019	Frontend + Documentation

1.10 Project deliverables

Here is a list of project deliverables for the Construction Material platform:

1. Project Proposal

- Purpose: Outlines the project goals, problem statement, stakeholders, objectives, scope and technology stack.
- Contents: Background, problem statement, objectives, scope, limitations, tools and work division.

2. Requirement Traceability Matrix (RTM)

- Purpose: Maps business requirements to technical requirements and design modules.
- Contents: BR-TR matrix, TR-Design matrix and full traceability matrix.

3. Software Design Specification (SDS)

- Purpose: Provides detailed design blueprint including diagrams, data models and architectural decisions.
- Contents: Class diagrams, sequence diagrams, ER diagram, MVC architecture and data dictionary.

4. Final Year Project Report (This Document)

- Purpose: Comprehensive documentation covering all chapters from introduction to conclusion.

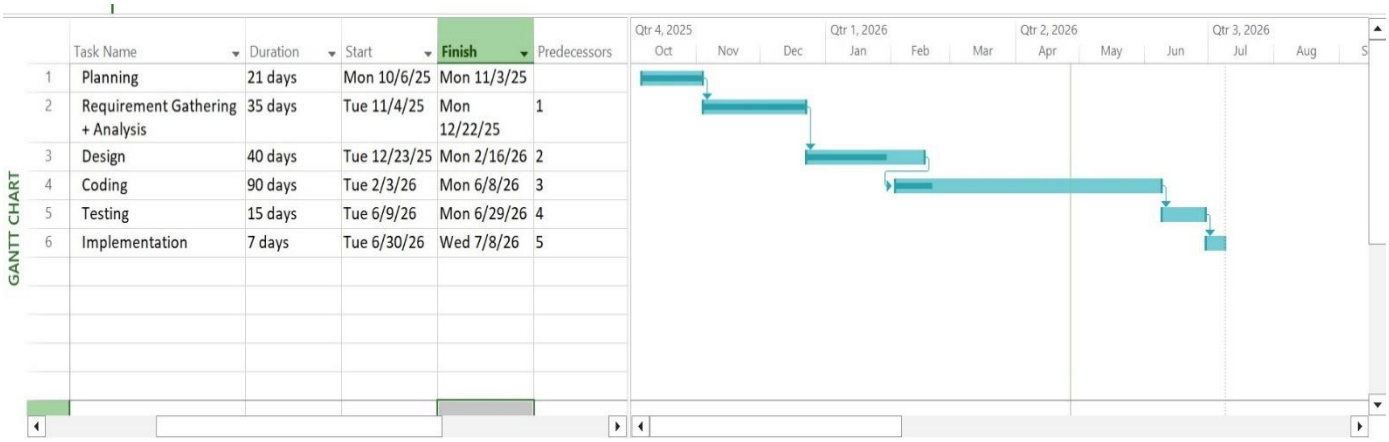
5. Working Web Application

- Purpose: Functional prototype demonstrating all core modules.
- Contents: Role-based dashboards, product listings, order management, payment, bidding and admin panel.

1.11 Project planning

Gantt chart representing the plans of carrying out the project activities, representing the milestones as well as resources required for each task:

Figure 1-1: Gantt Chart



1.12 Summary

The objective of this project is to build a web-based marketplace that enables both wholesale and retail transactions for construction materials in Pakistan. The platform will feature role-based modules, comprehensive product listings, streamlined order placement and tracking, bidding functionality, and secure payment with automated invoice generation. It will be developed using modern technologies including React.js, Node.js, Express.js, PostgreSQL, Sequelize, Stripe, JWT, and Nodemailer to ensure scalability, security, and efficiency. This solution addresses a critical gap in the market, as no existing platform in Pakistan currently integrates wholesale and retail services with supplier verification, bidding, and secure payment processing in a single unified system.

Chapter 2:

Analysis

2. Analysis

Construction Material is a web-based marketplace platform developed using React.js, Node.js, Express.js and PostgreSQL. It outlines the system's user classes, use cases, functional and nonfunctional requirements, and the rationale behind the technology selection.

2.1 User Classes and Characteristics

These are the user classes, and their characteristics are as follows:

Table 2 - 1: User classes & Characterization

User Class	Characteristics
Admin	Manages the whole system. Verifies and approves suppliers before their products are listed. Manages all users, products and orders. Generates system reports.
Supplier	Registers on the platform and waits for Admin verification. Once verified, uploads products and manages stock. Views incoming orders and submits bids for bulk orders.
Wholesaler	Register and log in. Buys and sells construction materials in bulk. Submits bulk order requests for bidding and selects the best seller bid.
Retailer	Register and log in. Buy and sell construction materials in small quantities. Places orders and makes secure online payments. Can also request a bid, but the condition is to be a verified retailer.
Customer	Register and logs in. Browse and searches products. Places orders, makes payments and tracks order status.

2.2 Requirement Identifying Techniques

There are fundamental approaches used to identify requirements. As gathering requirements requires meeting with stakeholders, the following approaches were used:

2.2.1 Interviews

Open interviews were conducted with stakeholders including construction material buyers, suppliers and wholesalers operating in local markets in Pakistan. These interviews helped gather practical pain points such as inconsistent pricing, lack of supplier verification, absence of digital invoicing and no support for bulk order bidding. The findings directly shaped the functional requirements of the platform.

2.2.2 Ethnography or Observation

Software systems do not exist in isolation. They are used in a social and organizational environment and software system requirements may be generated or constrained by that environment. Observation of existing manual practices in the construction material market was used to understand how buyers currently contact multiple sellers, compare prices manually and face delays in order processing. This helped derive requirements for a digital solution that replaces these inefficient processes.

2.2.3 Use Case and Scenarios

Developing use cases and scenarios to describe how the system will be used in different situations. This technique helps in understanding the interactions between users and the system, making it easier to identify and document functional requirements. Use cases are a way of describing interactions between users and a system using a graphical model and structured text.

2.3 Functional Requirements

This section describes the functional requirements of the system expressed in the natural language style. This section is organized by features as a system feature name and specific functional requirements associated with this feature. So, for each module and its feature the functional requirements are:

2.3.1 Functional Requirement 1

Table 2 - 2: Functional Requirement 1

Identifier	**FR-1**
Title	User Registration
Requirement	The system shall allow a new user to register by providing their name, email address, phone number, password and selecting a role. After successful registration, the user account is created and the user can log in.
Source	Business Requirement, Open Interviews
Rationale	Enables all stakeholders to create accounts and access role-specific features of the platform. Provides secure and personalized access.

Business Rule	Email must be unique. Password must be encrypted using Bcrypt.js. Role must be selected at registration. Supplier accounts require additional Admin verification before products can be listed.
Dependencies	None
Priority	High

2.3.2 Functional Requirement 2

Table 2 - 3: Functional Requirement 2

Identifier	**FR-2**
Title	User login and Session Management
Requirement	The system shall allow registered users to log in using their email and password. On successful login, a JWT token is issued to manage the session. The token is used to control role-based access to all protected routes and features.
Source	Business Requirement
Rationale	Provides secure access to the platform and ensures each user can only access features relevant to their assigned role.
Business Rule	Email and password must match the registered credentials. JWT token must be included in all protected API requests. Token expires after a set duration for security.
Dependencies	FR-1
Priority	High

2.3.3 Functional Requirement 3

Table 2 - 4: Functional Requirement 3

Identifier	**FR-3**
Title	Seller Verification by admin
Requirement	The system shall require newly registered sellers to be verified and approved by the admin before their product listings are visible on the platform. The admin can approve or reject a seller account from the Admin dashboard.
Source	Business Requirement

Rationale	Ensures only legitimate and trustworthy sellers can list products, reduce fraud risk and improving buyer confidence in the marketplace.
Business Rule	A seller account must have Admin approval status set to Approved before products can be listed. Rejected sellers cannot list products.
Dependencies	FR-1, FR-2
Priority	High

2.3.4 Functional Requirement 4

Table 2 - 5: Functional Requirement 4

Identifier	**FR-4**
Title	Product Listing and Inventory Management
Requirement	The system shall allow sellers to add, edit and delete product listings. Each listing must include product name, description, category, price, minimum wholesale quantity and stock level. Seller can upload product images using Multer. Stock levels must be updated in real time when orders are placed.
Source	Business Requirement, Open Interviews
Rationale	Provides buyers with accurate and up-to-date product information. Enables suppliers to manage their inventory digitally without manual processes.
Business Rule	Sellers can add or edit product listings. Stock level must not go below zero. Images are subject to Multer server-side file size restrictions.
Dependencies	FR-1, FR-2, FR-3
Priority	High

2.3.5 Functional Requirement 5

Table 2 - 6: Functional Requirement 5

Identifier	**FR-5**
Title	Product Browsing and Search
Requirement	The system shall allow all buyers to browse the full product catalog. Buyers shall be able to search and filter products by name, category and price. Product detail pages must display name, description, price, seller name, stock level and product images.
Source	Business Requirement, Open Interviews, Observation
Rationale	Allows buyers to find and compare construction materials easily on a single platform, removing the need to contact multiple sellers manually.

Business Rule	Only products from Admin-verified sellers are shown. Out-of-stock products are displayed with a stock unavailable indicator.
Dependencies	FR-1, FR-2, FR-4
Priority	High

2.3.6 Functional Requirement 6

Table 2 - 7: Functional Requirement 6

Identifier	**FR-6**
Title	Order Placement and Tracking
Requirement	The system shall allow authenticated buyers to place orders by selecting products and specifying quantity and shipping details. The system shall track the order status through the full lifecycle: Created, Pending, Paid, Completed and Cancelled. Buyers shall be able to view their full order history from their dashboard.
Source	Business Requirement, Open Interviews
Rationale	Enables buyers to place and track orders entirely online, replacing manual phonebased ordering and improving operational efficiency.
Business Rule	Only authenticated users can place orders. Order quantity must not exceed available stock. Order status transitions must follow the defined lifecycle sequence.
Dependencies	FR-1, FR-2, FR-4, FR-5
Priority	High

2.3.7 Functional Requirement 7

Table 2 - 8: Functional Requirement 7

Identifier	**FR-7**
Title	Secure Payment Processing and Invoice Generation
Requirement	The system shall process secure online payments through the Stripe payment gateway. Upon successful payment, the system shall automatically generate a digital PDF invoice containing the order summary and payment details. Buyers shall be able to download their invoice from the order history page.
Source	Business Requirement

Rationale	Provides a secure, transparent and paperless transaction process. Replaces manual cash-based payments and reduces fraud risk in construction material transactions.
Business Rule	Payments must be processed exclusively through Stripe. Invoice must be generated only after payment confirmation is received from Stripe. Online payments depend on third party gateway availability.
Dependencies	FR-1, FR-2, FR-6
Priority	High

2.3.8 Functional Requirement 8

Table 2 - 9: Functional Requirement 8

Identifier	**FR-8**
Title	Bidding for Bulk Orders
Requirement	The system shall allow Wholesalers & retailers to submit bulk order requests specifying the product, required quantity and any additional requirements. Verified Suppliers shall be able to view open bulk order requests and submit a bid amount. Wholesalers shall be able to view all submitted bids and accept the best offer, which will then initiate the order process. Suppliers shall be able to cancel the submitted bid.
Source	Business Requirement, Open Interviews
Rationale	Provides a competitive pricing mechanism for bulk construction material purchases. Allows buyers to get better prices by comparing multiple supplier bids instead of accepting a fixed price.
Business Rule	Only Admin-verified suppliers can submit bids. A wholesaler & retailer can only accept one bid per bulk order request. Once a bid is accepted, it cannot be modified.
Dependencies	FR-1, FR-2, FR-3, FR-6
Priority	High

2.3.9 Functional Requirement 9

Table 2 - 10: Functional Requirement 9

Identifier	**FR-9**
Title	Email Notifications
Requirement	The system shall send automated email notifications via Nodemailer to relevant users for the following events: order placement confirmation, order status updates, bid submission alerts to suppliers, bid result notification to wholesalers and price update alerts.
Source	Business Requirement, Observation
Rationale	Keeps all users informed about important events in real time without requiring them to manually check the platform, improving user experience and transaction transparency.
Business Rule	Notifications are sent only to verified and registered email addresses. Email delivery depends on the availability of the SMTP service through Nodemailer.
Dependencies	FR-1, FR-6, FR-7, FR-8
Priority	Medium

2.3.10 Functional Requirement 10

Table 2 - 11: Functional Requirement 10

Identifier	**FR-10**
Title	Admin Panel
Requirement	The admin shall be able to manage all system functionalities through the admin dashboard. This includes verifying and approving supplier accounts, managing all users (view, edit, delete), managing all products and orders, and generating system reports and analytics.
Source	Business Requirement, Observation
Rationale	Provides centralized control over the entire marketplace. Ensures platform integrity by controlling supplier verification and monitoring all transactions.
Business Rule	Admin credentials must be authenticated via JWT. Only users with the admin role can access the admin dashboard and perform administrative actions.

Dependencies	FR-1, FR-2
Priority	High

2.4 Non-Functional Requirements

These are constraints on the services or functions offered by the system. They include timing constraints, constraints on the development process, and constraints imposed by standards. Non-functional requirements often apply to the system rather than individual system features or services.

Following are non-functional requirements:

Table 2 - 12: Non-Functional Requirements

Identifier	Category	Requirement Description	Module	Priority
NFR-1	Performance	The system must respond to user requests within 3 seconds under normal traffic load.	All Modules	High
NFR-2	Performance	Product listings and search results must load within 5 seconds over a standard internet connection.	Module 2	High
NFR-3	Scalability	The system must support multiple concurrent users and be designed to accommodate future features like logistics tracking.	All Modules	High
NFR-4	Security	All passwords must be encrypted using Bcrypt.js. JWT tokens must be used for all authenticated API requests. Stripe must handle all payments.	Module 1, 4, 7	High
NFR-5	Usability	The UI must be responsive and follow consistent design guidelines across all user role dashboards.	All Modules	Medium
NFR-6	Responsiveness	The app layout must be adapted to all modern browsers and screen sizes including desktops, tablets and smartphones.	All Modules	High
NFR-7	Reliability	The system must use PostgreSQL transactions and Sequelize validation to ensure data consistency and prevent partial updates.	All Modules	High
NFR-8	Compatibility	The system must function smoothly across cross-platform devices and modern web browsers including Chrome, Edge and Firefox.	All Modules	Medium

NFR-9	Data Retention	All user data including orders, payments and invoices must be retained in PostgreSQL databases for at least one year.	Module 3, 4	Medium

2.5 Use Case Analysis

All systems involve interaction of some kind. This can be user interaction, which involves user inputs and outputs, interaction between the software being developed and other systems in its environment, or interaction between the components of a software system. User interaction modeling is important as it helps to identify user requirements.

2.5.1 Use Case 1

Table 2 - 13: Use Case 1

Use Case ID	**UC-1**
Use Case Name	User Registration
Actors	Customer, Retailer, Wholesaler, Supplier
Description	This use case allows a new user to create an account on the Construction Material platform by providing their personal details, setting a password and selecting their role.
Trigger	User opens the platform and selects the Register option on the landing page.
Preconditions	User must not already be registered with the same email address.
Postconditions	A new user account is created in the PostgreSQL Users DB. If the user is a Supplier, the account is marked as Pending until Admin approval. Other roles are immediately active.

Normal Flow	1. User opens the platform. 2. User selects Register. 3. User enters name, email, phone, password and selects role. 4. System validates the inputs. 5. System creates user account in Users DB. 6. If role is Supplier, account is flagged as Pending Verification. 7. User is redirected to the login page.
Alternative Flows	Email Already Exists: System shows an error message and prompts the user to use a different email or log in. Weak Password: If password does not meet criteria, user is prompted to correct it.
Business Rules	Email must be unique. Password must be encrypted using Bcrypt.js. Role must be selected at registration.
Assumptions	User has a stable internet connection. The platform is accessible via a modern browser.

2.5.2 Use Case 2

Table 2 - 14: Use Case 2

Use Case ID	**UC-2**
Use Case Name	User Login
Actors	Admin, Supplier, Wholesaler, Retailer, Customer
Description	This use case allows a registered user to log in to the platform using their email and password. A JWT token is issued and used to manage rolebased session access.
Trigger	User opens the platform and selects the Login option.
Preconditions	User must already be registered. Supplier accounts must have Admin approval status.
Postconditions	User is authenticated and redirected to their role-specific dashboard. JWT token is issued for session management.

Normal Flow	1. User opens the platform. 2. User navigates to the Login page. 3. User enters email and password. 4. System validates credentials against Users DB. 5. System issues a JWT token. 6. User is redirected to their role specific dashboard.
Alternative Flows	Invalid Credentials: System shows an error message and user remains on the login page. Pending Supplier: If supplier account is not yet verified, login is denied with a message to wait for Admin approval.
Business Rules	Email and password must match registered credentials. JWT token is required for all protected route access. Token expires after a set duration.
Assumptions	User has a stable internet connection. Firebase-equivalent JWT service is correctly configured.

2.5.3 Use Case 3

Table 2 - 15: Use Case 3

Use Case ID	**UC-3**
Use Case Name	Seller Verification
Actors	Admin, Seller
Description	This use case allows the Admin to review newly registered Seller accounts and either approve or reject them. Only approved Sellers can list products on the marketplace.
Trigger	A new Seller registers on the platform. Admin opens the Seller Verification section of the Admin dashboard.
Preconditions	A Seller account with Pending status must exist in the Users DB. Admin must be logged in.
Postconditions	Seller account status is updated to Approved or Rejected in Users DB. Approved seller can now add product listings.

Normal Flow	1. Seller registers on the platform. 2. Account is flagged as Pending Verification. 3. Admin logs in to the Admin dashboard. 4. Admin opens Seller Verification section.
	5. Admin reviews the seller details. 6. Admin clicks Approve. 7. System updates seller status to Approved in Users DB. 8. Seller is notified via email.
Alternative Flows	Admin Rejects: Admin clicks Reject. System updates status to Rejected. Seller is notified and cannot list products.
Business Rules	Only Admin can approve or reject supplier accounts. Sellers without Approved status cannot add product listings.
Assumptions	Admin has access to the Admin dashboard. Email notification service is operational.

2.5.4 Use Case 4

Table 2 - 16: Use Case 4

Use Case ID	**UC-4**
Use Case Name	Manage Product Listings
Actors	Seller
Description	This use case allows a verified Seller to add new product listings and manage existing ones. The Seller can edit product details, update stock levels and upload product images.
Trigger	Verified Supplier logs in and navigates to Product Management on the Seller dashboard.
Preconditions	Seller must be logged in and have Admin-Approved status.

Postconditions	Product is added, updated or deleted in the Products DB. Stock level is updated accordingly.
Normal Flow	1. Seller logs in. 2. Seller opens Product Management dashboard. 3. Seller clicks Add Product. 4. Seller enters product name, description, category, price, min wholesale quantity and stock. 5. Seller uploads product image via Multer. 6. System validates the inputs. 7. Product is saved to Products DB.
Alternative Flows	Edit Product: Seller selects an existing product and updates the details. Delete Product: Seller removes a product listing from the marketplace.
Business Rules	Only Admin-verified suppliers can add or edit products. Stock level must not go below zero. Image uploads are subject to Multer file size restrictions.
Assumptions	Seller has correct product information and uses genuine product images.

2.5.5 Use Case 5

Table 2 - 17: Use Case 5

Use Case ID	**UC-5**
Use Case Name	Browse and Search Products
Actors	Wholesaler, Retailer, Customer
Description	This use case allows authenticated buyers to browse the product catalog, search by name or category and view detailed product information before placing an order.
Trigger	Buyer logs in and navigates to the Marketplace or Product Listing page.
Preconditions	Buyer must be authenticated. Products from verified suppliers must be available in the Products DB.

Postconditions	Buyer views product listing. Buyer can proceed to place an order from the product detail page.
Normal Flow	1. Buyer logs in. 2. Buyer navigates to the Marketplace. 3. System loads all verified supplier product listings from Products DB. 4. Buyer enters a search term or selects a category filter. 5. System displays filtered product results. 6. Buyer clicks on a product card. 7. Product detail page is shown with name, description, price, stock and supplier.
Alternative Flows	No Results Found: System displays a No products found message and suggests adjusting the search filters. Out of Stock: Product is displayed with a Stock Unavailable label.
Business Rules	Only products from Admin-verified suppliers are shown. Price comparison is available for the same product from multiple suppliers.
Assumptions	Buyer has a stable internet connection.

2.5.6 Use Case 6

Table 2 - 18: Use Case 6

Use Case ID	**UC-6**
Use Case Name	Place Order
Actors	Wholesaler, Retailer, Customer
Description	This use case allows authenticated buyers to place an order for selected products by specifying quantity and shipping details. The order is created in the Orders DB with a status of Pending.
Trigger	Buyer clicks Place Order on the product detail page or cart page.
Preconditions	Buyer must be authenticated. Selected product must be in stock.

Postconditions	Order is created in Orders DB with status Created. Stock level is reserved. Supplier receives a new order notification.
Normal Flow	1. Buyer selects a product and clicks Add to Cart. 2. Buyer reviews cart and enters shipping details. 3. Buyer clicks Place Order. 4. System checks product stock availability in Products DB. \5. System creates an order
Alternative Flows	Insufficient Stock: System shows an error and prevents order placement. Order Cancelled: Buyer or Admin cancels the order and status is updated to Cancelled.
Business Rules	Only authenticated users can place orders. Order quantity must not exceed available stock. Order status must follow the defined lifecycle.
Assumptions	Buyer has entered correct shipping details.

2.5.7 Use Case 7

Table 2 - 19: Use Case 7

Use Case ID	**UC-7**
Use Case Name	Make Payment and Download Invoice
Actors	Wholesaler, Retailer, Customer, Stripe
Description	This use case allows the buyer to complete secure online payment for their placed order through Stripe. On successful payment, the system generates a digital PDF invoice and makes it available for download.
Trigger	Buyer clicks Proceed to Payment on the order summary page.
Preconditions	An order must exist in Orders DB with status Pending. Stripe payment gateway must be available.
Postconditions	Payment is processed and recorded in Payments DB. Order status is updated to Paid. PDF invoice is generated and stored. Buyer receives invoice download link.

Normal Flow	1. Buyer clicks Proceed to Payment. 2. Stripe payment form is shown. 3. Buyer enters card details and submits. 4. System sends payment request to Stripe. 5. Stripe confirms the transaction. 6. System records the payment in Payments DB.
	7. System updates order status to Paid in Orders DB. 8. System generates PDF invoice. 9. Buyer is shown the invoice download button.
Alternative Flows	Payment Failed: Stripe returns a failure response. System shows an error and order status remains Pending. Buyer can retry.
Business Rules	All payments must be processed through Stripe. Invoice is generated only after Stripe payment confirmation. Online payments depend on third party gateway availability.
Assumptions	Stripe services are operational. Buyer has a valid payment method.

2.5.8 Use Case 8

Table 2 - 20: Use Case 8

Use Case ID	**UC-8**
Use Case Name	Submit Bulk Order Request for Bidding
Actors	Seller (Wholesaler, retailer)
Description	This use case allows a seller to submit a bulk order request specifying the product, required quantity and any additional requirements. Verified sellers can then view the request and submit their bid amounts.
Trigger	Seller clicks Request Bulk Order on the product listing or bidding page.
Preconditions	Seller must be authenticated. Product must exist in the Products DB.

Postconditions	Bulk order request is saved in Bids DB. Verified suppliers are notified of the new request via Nodemailer.
Normal Flow	1. Seller selects a product. 2. Seller clicks Request Bulk Order. 3. Seller enters required quantity and any notes. 4. System saves the bulk order request in Bids DB. 5. System sends email notification to all verified suppliers for the product category via Nodemailer.
Alternative Flows	No Response from Suppliers: Seller can cancel the request or resubmit.
Business Rules	Only authenticated sellers can submit bulk order requests. Notifications depend on Nodemailer availability.
Assumptions	Wholesalers have a stable internet connection.

2.5.9 Use Case 9

Table 2 - 21: Use Case 9

Use Case ID	**UC-9**
Use Case Name	Submit and Manage Bids
Actors	Supplier, Wholesaler
Description	This use case allows a verified Supplier to view open bulk order requests and submit a bid with a proposed price and quantity. The Wholesaler can then view all submitted bids and accept the best offer. The accepted bid initiates the order process.
Trigger	Supplier opens the Bidding section on the Supplier dashboard. Wholesaler opens the My Bids section on the Wholesaler dashboard.
Preconditions	Supplier must be Admin-verified. A bulk order request must exist in Bids DB.

Postconditions	Bid is saved in Bids DB. If bid is accepted, order process is initiated. Supplier is notified of the acceptance.
Normal Flow	1. Supplier opens the Bidding section. 2. System loads open bulk order requests from Bids DB. 3. Supplier selects a request and submits a bid amount. 4. Bid is saved in Bids DB. 5. Wholesaler views all bids for their request. 6. Wholesaler selects and accepts the best bid. 7. System creates an order from the accepted bid. 8. System notifies the winning supplier via Nodemailer.
Alternative Flows	Supplier Cancels Bid: Supplier can cancel a submitted bid before it is accepted. No Bids Received: Wholesaler sees a No Bids Yet message.
Business Rules	Only Admin-verified suppliers can submit bids. A wholesaler can only accept one bid per request. Once accepted, a bid cannot be modified.
Assumptions	Email notification service is operational.

2.5.10 Use Case 10

Table 2 - 22: Use Case 10

Use Case ID	**UC-10**
Use Case Name	Track Order Status
Actors	Wholesaler, Retailer, Customer
Description	This use case allows a buyer to view their order history and track the current status of each order through the full lifecycle: Created, Pending, Paid, Completed and Cancelled.
Trigger	Buyer clicks Order History on their dashboard.
Preconditions	Buyer must be authenticated. At least one order must exist in Orders DB.
Postconditions	Buyer views updated order status and can download invoice for paid orders.

Normal Flow	1. Buyer navigates to Order History on the dashboard. 2. System fetches all orders for the buyer from Orders DB. 3. Buyer views list of orders with status and date. 4. Buyer clicks on a specific order to see details. 5. If order is Paid, buyer clicks Download Invoice to download the PDF.
Alternative Flows	No Orders: System displays No orders placed yet message.
Business Rules	Invoice download is only available for orders with Paid status.
Assumptions	Buyer has placed at least one order.

2.5.11 Use Case 11

Table 2 - 23: Use Case 11

Use Case ID	**UC-11**
Use Case Name	Admin User Management
Actors	Admin
Description	This use case allows the Admin to view, manage and delete user accounts across all roles from the Admin dashboard.
Trigger	Admin opens the User Management section of the Admin dashboard.
Preconditions	Admin must be authenticated.
Postconditions	User record is updated or deleted in the PostgreSQL Users DB.
Normal Flow	1. Admin logs in to the Admin dashboard. 2. Admin selects User Management. 3. System loads all registered users from Users DB. 4. Admin can view, edit or delete any user account. 5. System updates the Users DB accordingly.
Alternative Flows	Database Operation Failure: System displays an error message.

Business Rules	Sensitive actions such as deletion require confirmation. Only Admin can perform user management actions.
Assumptions	PostgreSQL database services are operational.

2.5.12 Use Case 12

Table 2 - 24: Use Case 12

Use Case ID	**UC-12**
Use Case Name	Admin Product Management
Actors	Admin
Description	This use case allows the Admin to view, edit and remove product listings from the platform. Admin can also flag inappropriate or incorrect listings.
Trigger	Admin opens the Product Management section of the Admin dashboard.
Preconditions	Admin must be authenticated.
Postconditions	Product record is updated or removed from the Products DB.
Normal Flow	1. Admin opens Product Management. 2. System loads all product listings from Products DB. 3. Admin can view, edit or delete any product. 4. System updates the Products DB.
Alternative Flows	Database Operation Failure: System shows an error message.
Business Rules	Sensitive actions require confirmation. Only Admin can remove or edit supplier product listings.
Assumptions	PostgreSQL database services are operational.

2.5.13 Use Case 13

Table 2 - 25: Use Case 13

Use Case ID	**UC-13**
Use Case Name	Admin Order Management
Actors	Admin

Description	This use case allows the Admin to view all orders across all users, monitor order statuses and intervene to cancel or update orders if needed.
Trigger	Admin opens the Order Management section of the Admin dashboard.
Preconditions	Admin must be authenticated.
Postconditions	Order status is updated in the Orders DB.
Normal Flow	1. Admin opens Order Management. 2. System loads all orders from Orders DB. 3. Admin views order details including buyer, supplier, status and amount. 4. Admin can update order status or cancel an order. 5. System updates the Orders DB.
Alternative Flows	Database Operation Failure: System shows an error message.
Business Rules	Only Admin can force-cancel or modify orders outside the normal buyerinitiated flow.
Assumptions	PostgreSQL database services are operational.

2.5.14 Use Case 14

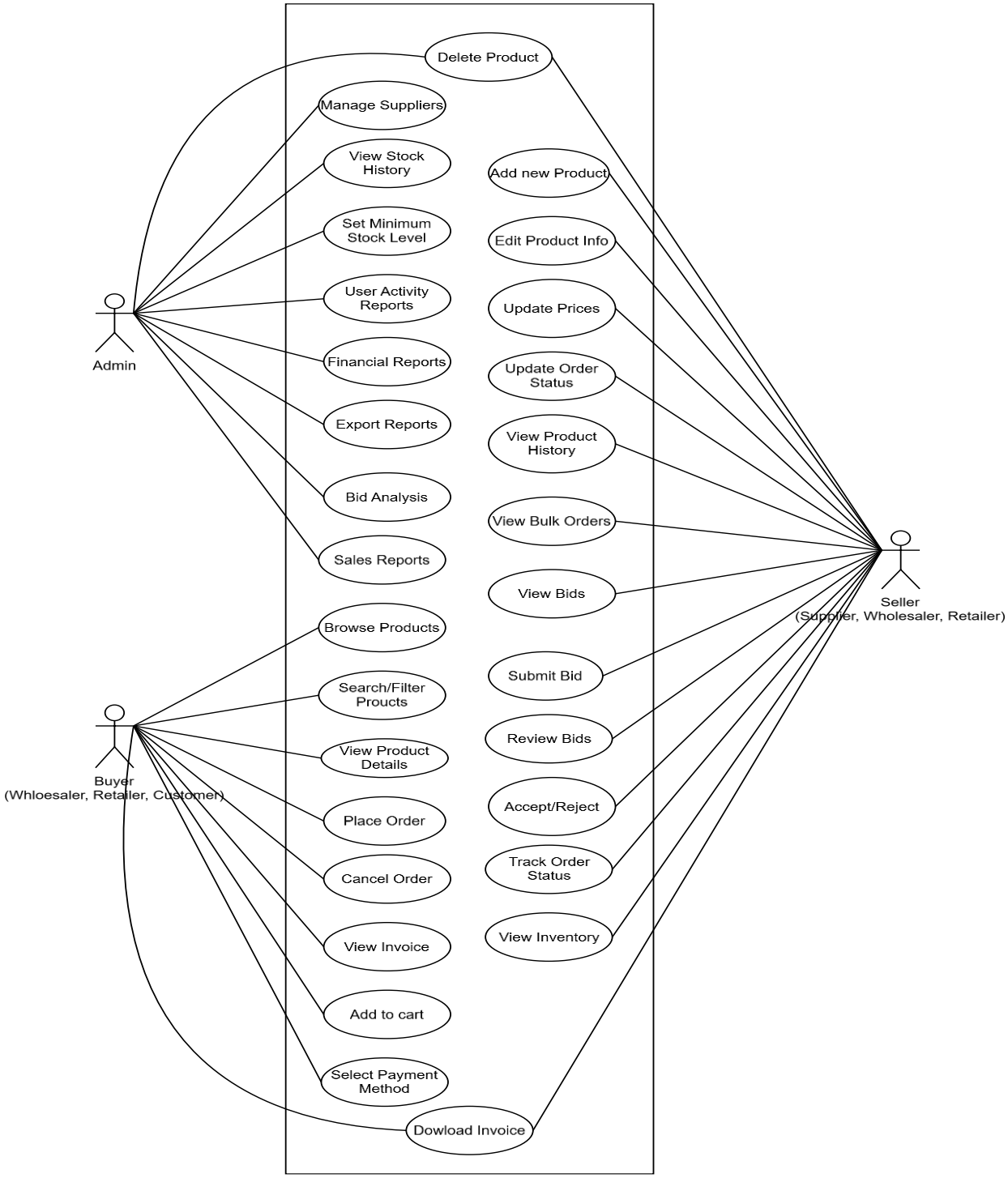
Table 2 - 26: Use Case 14

Use Case ID	**UC-14**
Use Case Name	Generate System Reports
Actors	Admin
Description	This use case allows the Admin to generate reports and view analytics about platform activity including total orders, total revenue, user registrations and supplier performance.
Trigger	Admin opens the Reports section of the Admin dashboard.

Preconditions	Admin must be authenticated. Data must exist in the PostgreSQL databases.
Postconditions	Reports and analytics are displayed on the Admin dashboard.
Normal Flow	1. Admin opens the Reports section. 2. System queries data from relevant PostgreSQL databases. 3. System displays summary reports including orders, revenue and user activity. 4. Admin can view detailed breakdowns per time period.
Alternative Flows	No Data Available: System shows a No data to display message.
Business Rules	Reports are generated in real time from the database. Only Admin has access to the Reports section.
Assumptions	PostgreSQL database services are operational. Sufficient data exists in the system.

2.5.15 Use Case Diagram

Figure 2-1: Use Case Diagram



2.6 External Interface Requirements

The system's external interface requirements encompass user, software, and API interfaces. The User Interface (UI) must support modern web browsers, featuring a responsive design that functions seamlessly across desktops, tablets, and mobile devices. For Software Interfaces, the system relies on Stripe for payment processing, Nodemailer for sending email notifications and Multer to handle product image uploads. Additionally, all client-server interactions are facilitated through API Interfaces, utilizing RESTful APIs built with Node.js and Express.js.

2.6.1 User Interface Requirements

Here is a detailed description of the logical characteristics of each user interface that the system needs

1. General UI Requirement:
    - Accessible design following basic accessibility standards such as colour contrast, font size and labels.
    - Built with React.js and Bootstrap for a consistent and responsive layout.
2. Customer / Retailer / Wholesaler Interface:
    - Login and Register page.
    - Product listing and search page.
    - Order placement and payment page.
    - Order history and tracking page.
    - Bid submission and comparison page.

3.Supplier Interface:

- Login and Register page.
- Product management dashboard.
- Incoming orders page.
- Bid submission page.

4.Admin Interface:

- Admin login page.
- Admin dashboard with user management, product management and reports.
- Seller verification page.

5. Notifications:

- Email notifications for order status, bid results and price changes.

2.6.2 Software Interfaces

The software interface connections are divided into database management, frontend frameworks, and backend services. The primary Database Management System is PostgreSQL, utilizing the Sequelize ORM. PostgreSQL was specifically chosen because the platform operates as a marketplace requiring multiple separate databases one for each service domain (users, products, orders, payments, and bids)which it handles efficiently through its multi-schema and multi-database support capabilities. For Frontend Frameworks and Libraries, the application uses React.js to deliver a component-based web frontend, supported by Bootstrap and CSS for responsive styling and layout. Axios is implemented to handle API requests and HTTP communication between the frontend and backend. The Backend Services are built with Node.js using Express.js. This includes RESTful API endpoints that drive all core features, including product management, order management, payment processing, and user authentication.

2.6.3 Hardware Interfaces

The software-hardware interface requirements dictate the supported devices and hosting environments necessary for system operations. The Supported Hardware Device Types include desktop computers and laptops, which provide administrators, suppliers, and buyers with full, comprehensive access to the system. Tablets and smartphones are also fully supported through a responsive design optimized for browsing and order placement. Regarding Hosting and Server Deployment, the system requires a cloud server to host both the backend and the database, running on an operating system equipped with a Node.js runtime environment.

2.6.4 Communications Interfaces

The system's communication interfaces outline necessary browser compatibility and networking standards. To meet Web Browser Requirements, the application must be fully functional on modern browsers, specifically Google Chrome, Microsoft Edge, and Mozilla Firefox. For Network Protocols, all communications must be securely transmitted over HTTPS to ensure data encryption. The architecture utilizes RESTful API endpoints for all client-server interactions, while internal data exchanges strictly follow JSON as the standard data format. Finally, email communications are executed using the SMTP protocol through Nodemailer.

2.7 Summary

This chapter outlines the process of identifying, eliciting and analyzing system requirements for the Construction Material platform. Users are divided into five main categories: Admin, Supplier, Wholesaler, Retailer and Customer. To gather requirements, techniques such as open interviews, ethnographic observation and use case analysis were applied. The collected requirements are then classified into ten functional requirements and nine non-functional requirements, supported by fourteen detailed use case descriptions and a use case diagram.

Chapter 3:
System Design

3. System Design

Our system is a web-based marketplace that connects suppliers, wholesalers, retailers and normal customers. It provides an online single platform to buy and sell construction materials. It also handles product listings, stock management, orders, bidding for bulk orders, payment processing and notifications.

3.1 Design Considerations

3.1.1 Dependencies and interactions

- These are some dependencies and interactions with other systems:
- Web Server: It handles routes, APIs and systems logic.
- Database (MySQL): It stores users, products, orders and payments.
- Authentication System (JWT): It provides security to our system.
- Payment Gateway (Stripe): It is used for secure payments.
- Email Service (Nodemailer): It is used for Sending Emails of price updating, feed updating etc.
- Image Service (Multer): It handles photo uploads.

3.1.2 Design Constraints

Our web-app must fulfill the design constraints:

- Performance: Our system must respond quickly to users during heavy traffic with responsive UI and low latency.
- Scalability: It is designed to accommodate future features like logistics tracking.
- Security: Secure authentication, encrypted transactions and role-based access are control with strict security rules.
- Compatibility: Our system must function smoothly across cross-platform devices.

3.1.3 Assumptions and Risks

Assumptions:

- Users have stable internet connections to access.
- Suppliers provide correct product information and use real pictures.
- Admin responsibility to verify suppliers before their product listed.
- Suppliers maintain the correct stock while making the listings.

Risks:

- Suppliers may upload fake or incorrect product information.
- Payment API and live order testing may take time.
- Slow user internet can result in incomplete page loads or API failures.
- The system goes down during any server or database downtime.

Limitations:

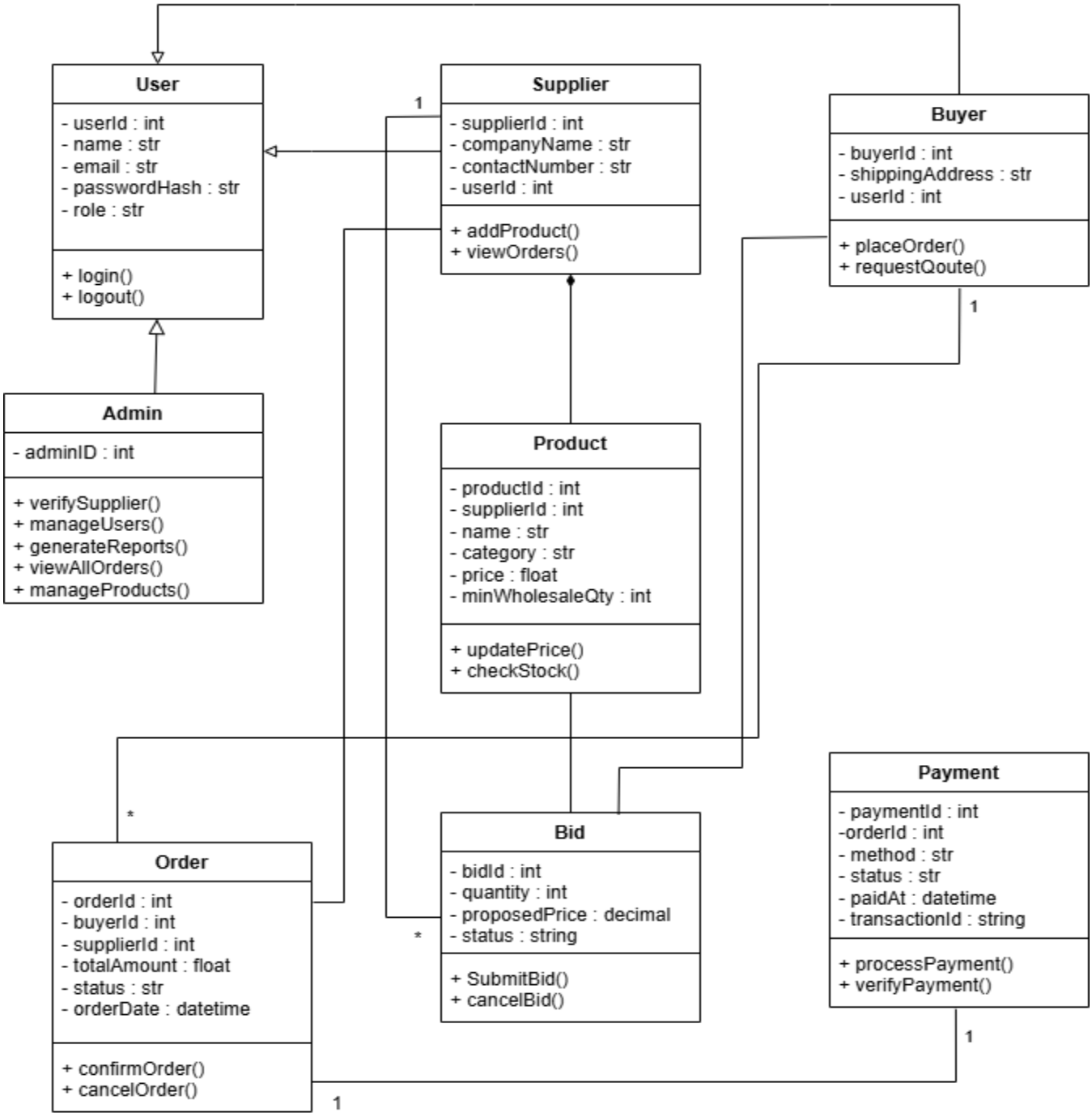
- Real-time delivery tracking is not a part of current scope.
- Online payments depend on third party gateways.
- Image uploads are limited by server and Multer file size restrictions. Design Models

3.2 Design Models

The class diagram represents a straightforward platform for construction materials with clear role separation. The User class serves as the foundation, extending into specialized Supplier and Buyer roles that define distinct user responsibilities. Suppliers manage Product listings and track stock through Inventory, while Buyers create Orders for purchases. The Order processing is completed via Payment transactions, creating a clean workflow from product selection to purchase completion. This small architecture effectively captures the essential buyer-supplier relationship in a marketplace with focused classes for core functionality.

3.2.1 Class Diagram

Figure 3 - 1: Class Diagram



3.2.2 Sequence Diagram

Figure 3 - 2: Sequence Diagram 1

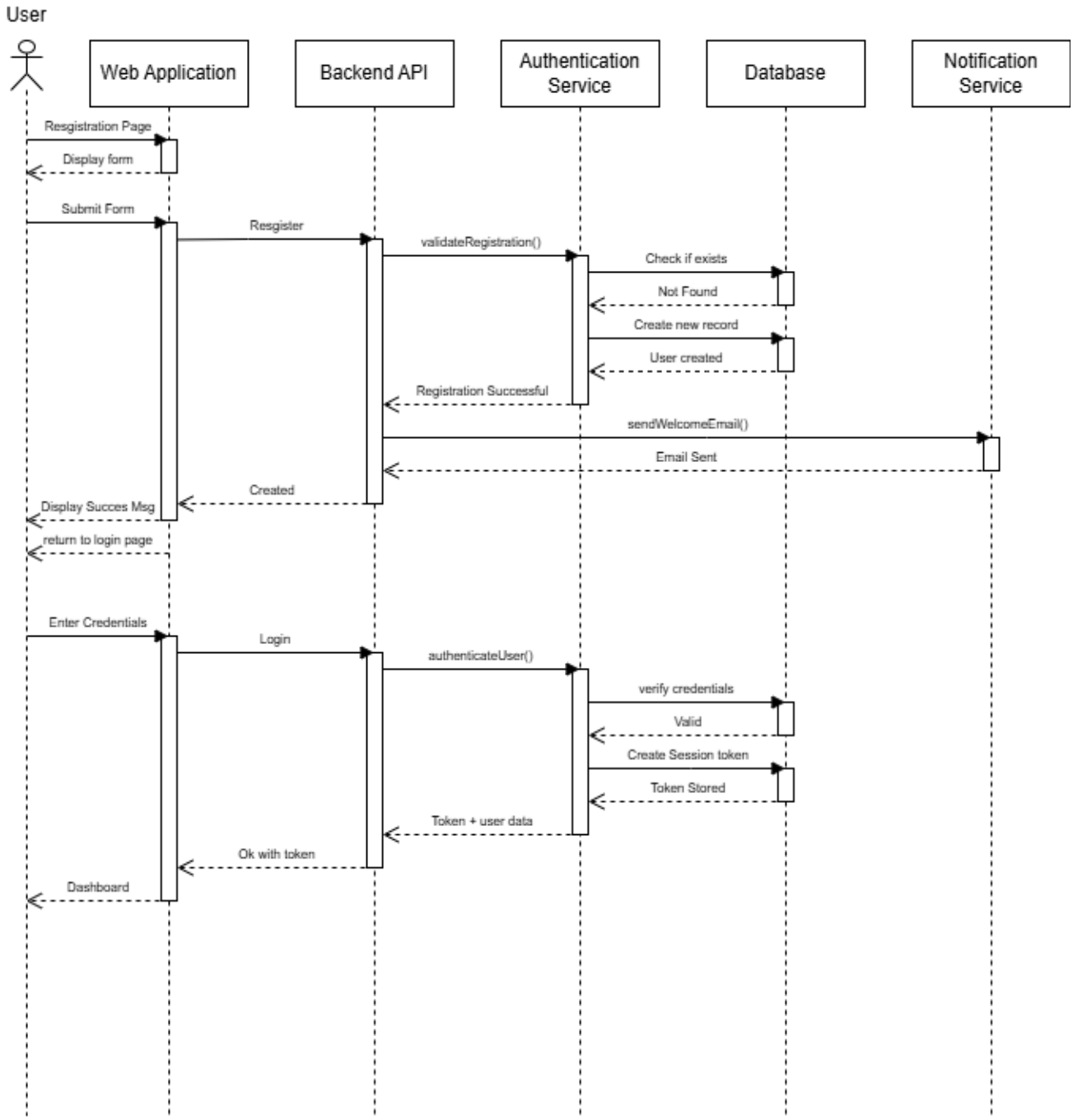


Figure 3 - 3: Sequence Diagram 2

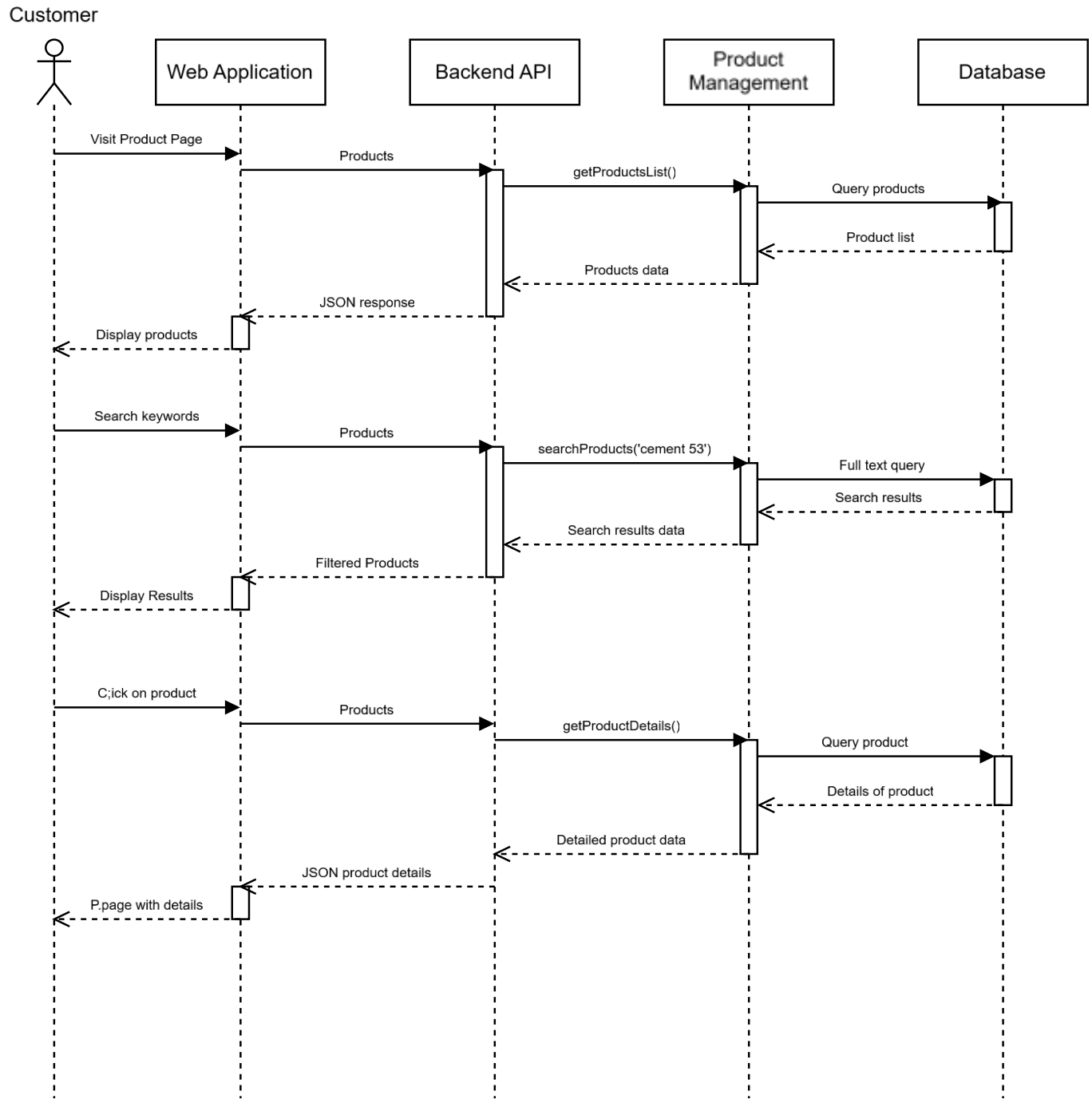


Figure 3 - 4: Sequence Diagram 3

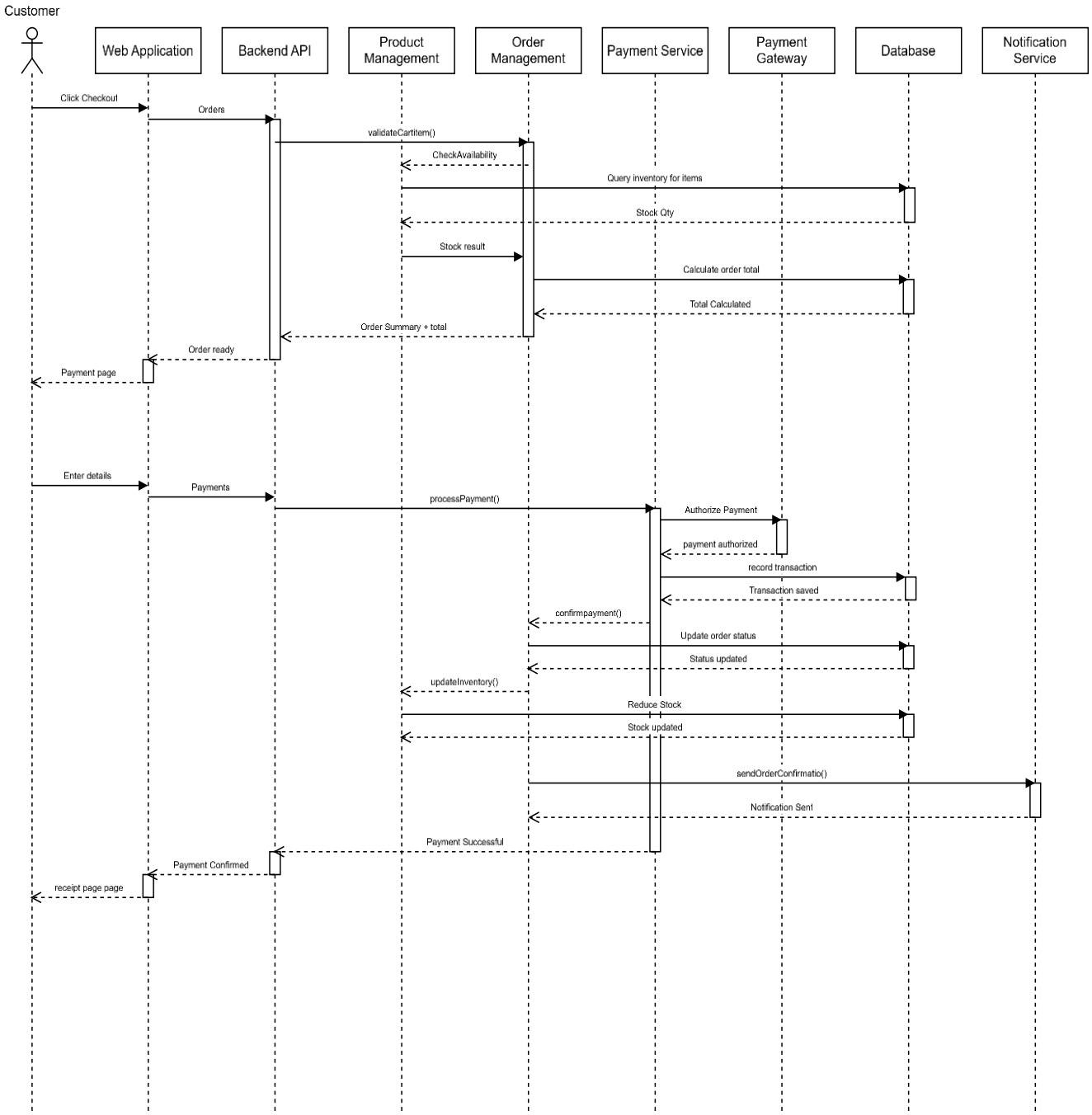


Figure 3 - 5: Sequence Diagram 4

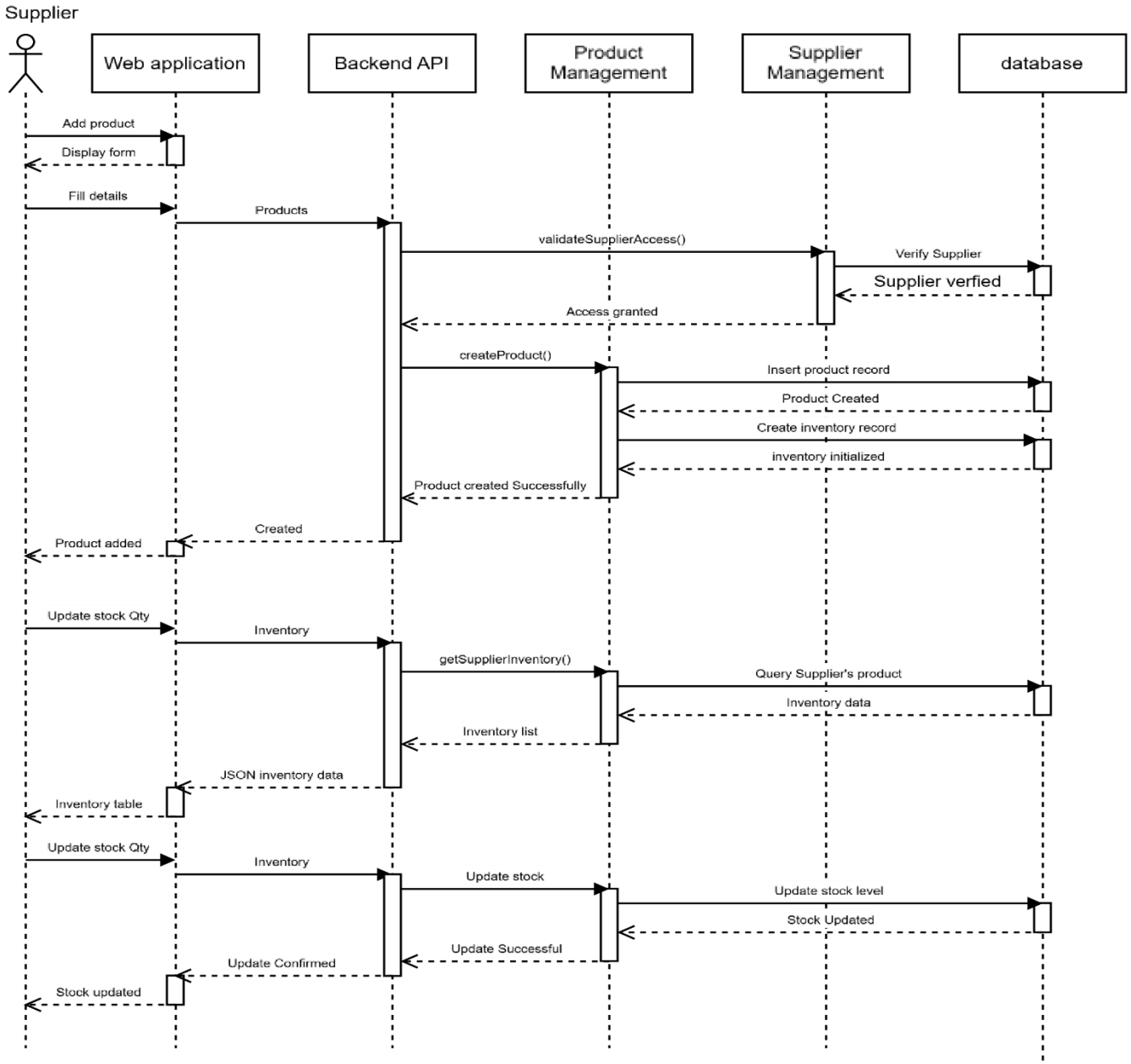
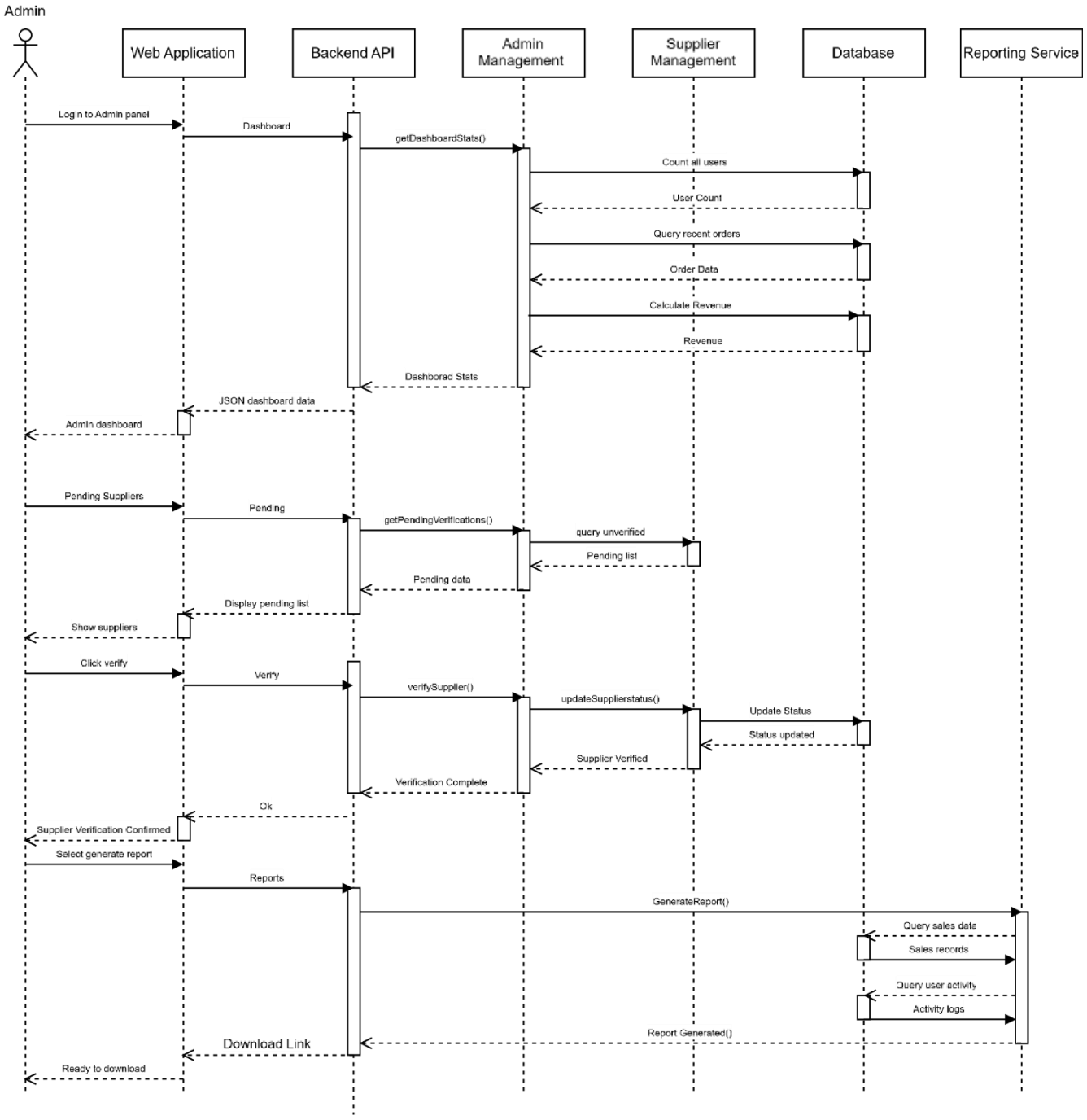
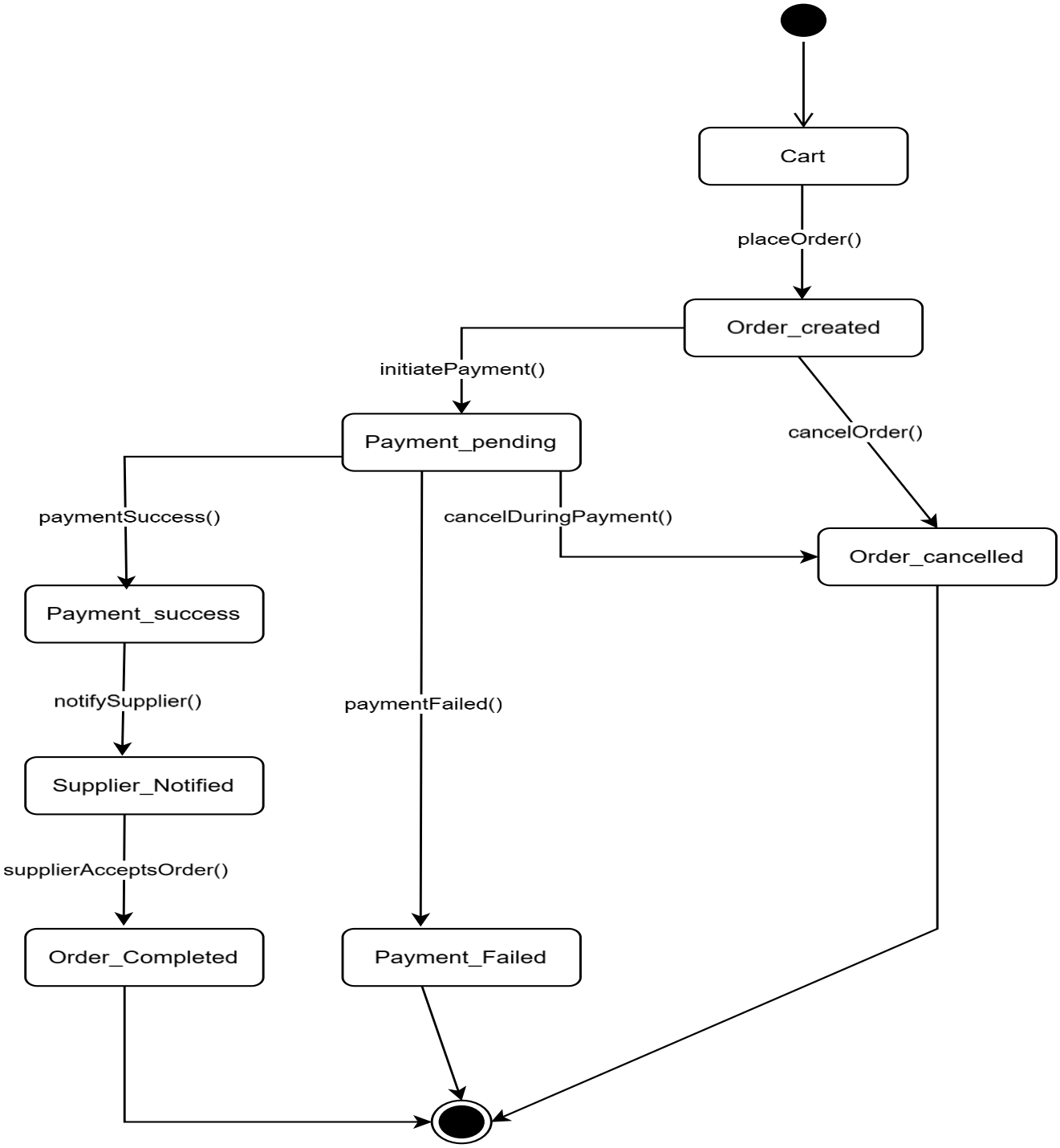


Figure 3 - 6: Sequence Diagram 5



3.2.3 State Transition Diagram

Figure 3 - 7: State Transition Diagram

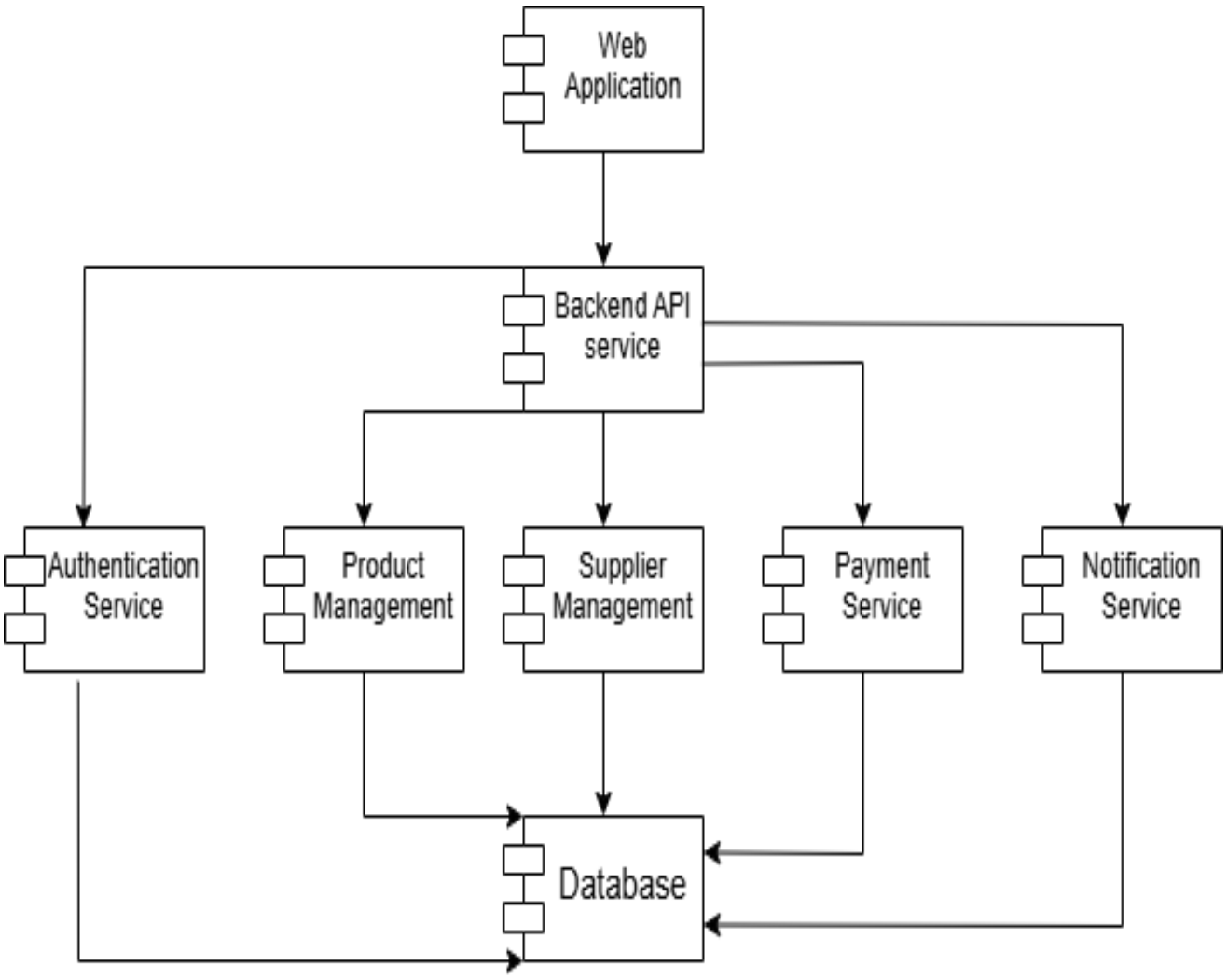


3.3 Architectural Design

Architectural design refers to the overall structure and organization of construction material system. It defines how all the components user interfaces, business logic modules, databases and external services work together as a cohesive system to meet project's functional and non-functional requirements, ensuring it is scalable, maintainable, and reliable for real-world use.

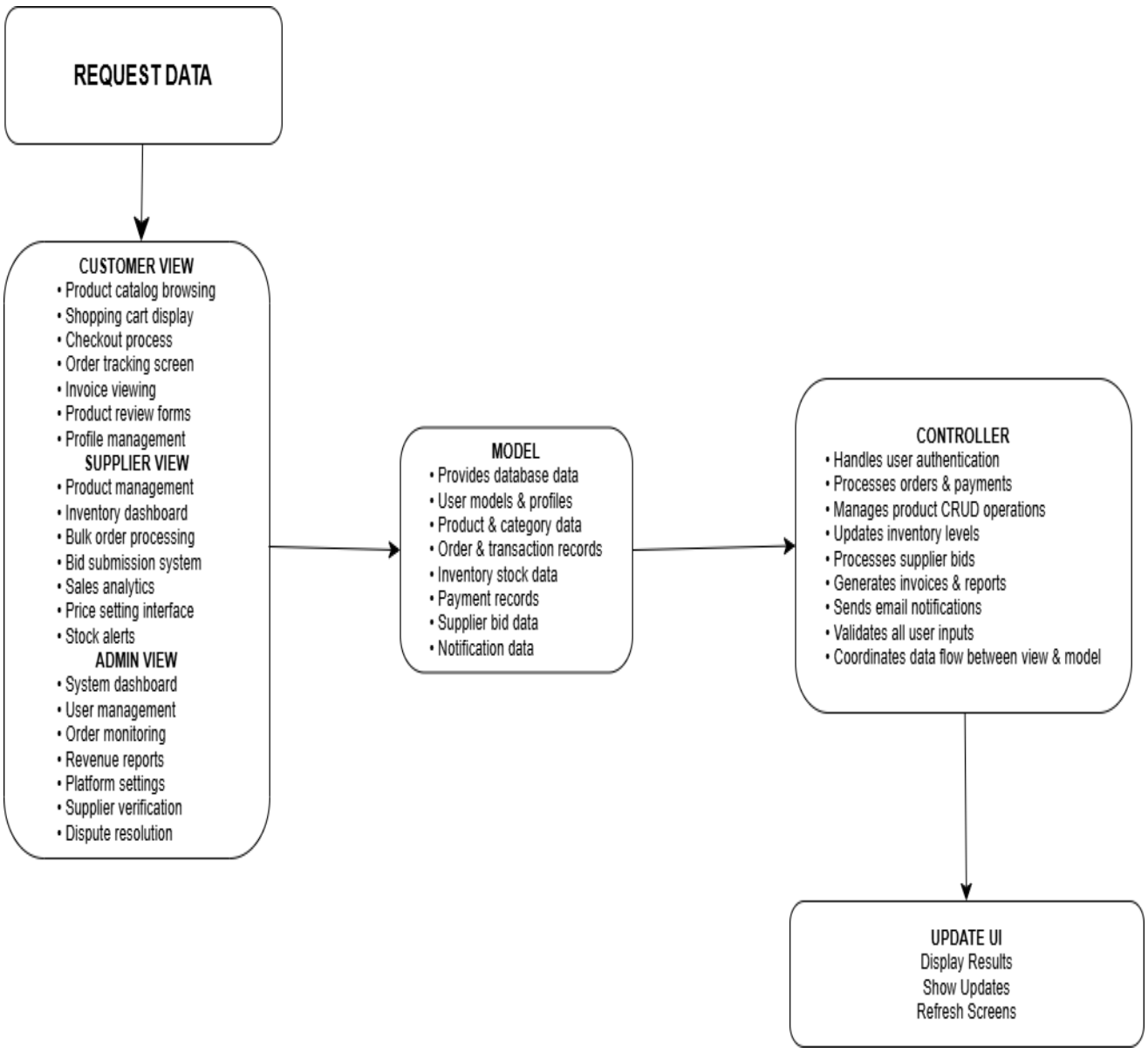
3.3.1 UML Component Diagram

Figure 3 - 8: UML Component Diagram



3.3.4 MVC Model

Figure 3 - 9: MVC Model



3.4 Data Design

Our database follows a relational model with tables.

1. Users:

- User_id
- Name
- Email
- Role • Password_hash **2 Products:**
- Products_id
- Name
- Description
- Price
- Stock • Supplier_id **3 Orders:**
- Order_id
- User_id
- Total
- Status
- Payments_id

4 Invoices:

- Invoice_id
- Order_id
- Pdf_url • Generated_at

5 Bids:

- Bid_id
- Order_id
- Supplier_id
- Bid_amount
- Status

Relationships:

- **One Supplier → Many Products** (A supplier can list many products)
- **One Customer → Many Orders** (A customer can place multiple orders)
- **One Order → Many Order Items** (Each order contains multiple products)
- **One Product → One Inventory** (Each product has stock information)
- **One Order → One Payment** (Each order has one payment record) **DataFlow:**

When a customer places an order:

- System checks product availability in inventory
- Creates an order record in database
- Processes payment transaction
- Updates inventory stock
- Sends confirmation notifications

This design ensures data consistency, fast performance and easy scaling as the platform grows.

3.4.1 Data Dictionary

Terminology	Description
Admin	A role with full system access and verification privileges.
User	It represents all participants with roles
Supplier	It represents verified suppliers who lists and sell materials
Buyer	It represents customer who purchase materials
Product	It represents construction material available for sale
Inventory	End user purchasing material in retail.
Order	It represents a purchase.
Bid	It represents a supplier bid amount for bulk order

Payment	It represents transaction record for order processing.
Invoice	It contains payment details and order summmary
Notification	It represents email alerts for order / bids etc

Object Oriented Approach

User:

Attributes:

- userId
- name
- email
- passwordHash
- role

Methods with parameters:

- Login(email, password)
- Logout()

Admin:

Attributes:

- adminID

Methods with parameters:

- verifySupplier(supplierId)
- manageUsers(action, userId)
- generateReports(startDate, endDate)
- viewAllOrders()
- manageProducts(productId, action)

Supplier:

Attributes:

- supplierId
- companyName
- contactNumber
- userId

Methods:

- addProduct(name, price, category, minWholesaleQty)
- viewOrders()

Buyer:

Attributes:

- buyerId
- ShippingAddress
- userId

Methods:

- placeOrder(items, shippings address)
- requestQoute(productId, quantity)

Product:

Attributes:

- productId
- supplierId
- name category
- price
- minWholesaleQty

Methods:

- updatePrice(newPrice)
- checkStock()

Order:

Attributes:

- orderId
- buyerId
- supplierId
- totalAmount
- status
- orderDate

Methods:

- confirmOrder()
- cancelOrder(reason)

Payment:

Attributes:

- paymentId
- orderId
- method
- status
- paidAt
- transactionId **Methods:**
- processPayment(amount, method) verifyPayment(transactionId)

Bid:

Attributes:

- bidId

- quantity
- proposedPrice
- status

Methods:

- submitBid(supplierId, orderId, Qty)
- cancelBid(reason)

3.5 User Interface Design

Here is an overview of the functionality of the Construction Material platform from the user's perspective:

User Roles

- **Customer / Retailer / Wholesaler:** A buyer can register, log in, browse products, place orders, make online payments, download invoices, submit bulk order requests and compare supplier bids.
- **Supplier:** A supplier can register and wait for Admin verification, then add and manage product listings, view incoming orders and submit bids for bulk order requests.
- **Admin:** An Admin can log in, verify supplier accounts, manage all users, products, orders and generate system reports.

Buyer Workflow

- **Registration:** The buyer registers by providing name, email, password and role.
- **Login:** The buyer logs in using their credentials.
- **Browse Products:** The buyer searches and filters products by category, price and supplier.
- **Place Order:** The buyer selects products, enters quantity and shipping details and places an order.
- **Make Payment:** The buyer pays securely through Stripe and receives a digital invoice.
- **Track Order:** The buyer tracks order status from their order history page.

Supplier Workflow

- **Registration:** The supplier registers and waits for Admin verification.
- **Login:** The supplier logs in once verified.
- **Manage Products:** The supplier adds, edits and manages product listings.

- **View Orders:** The supplier views and manages incoming orders.
- **Submit Bid:** T he supplier submits a bid amount for bulk order requests.

Feedback Information

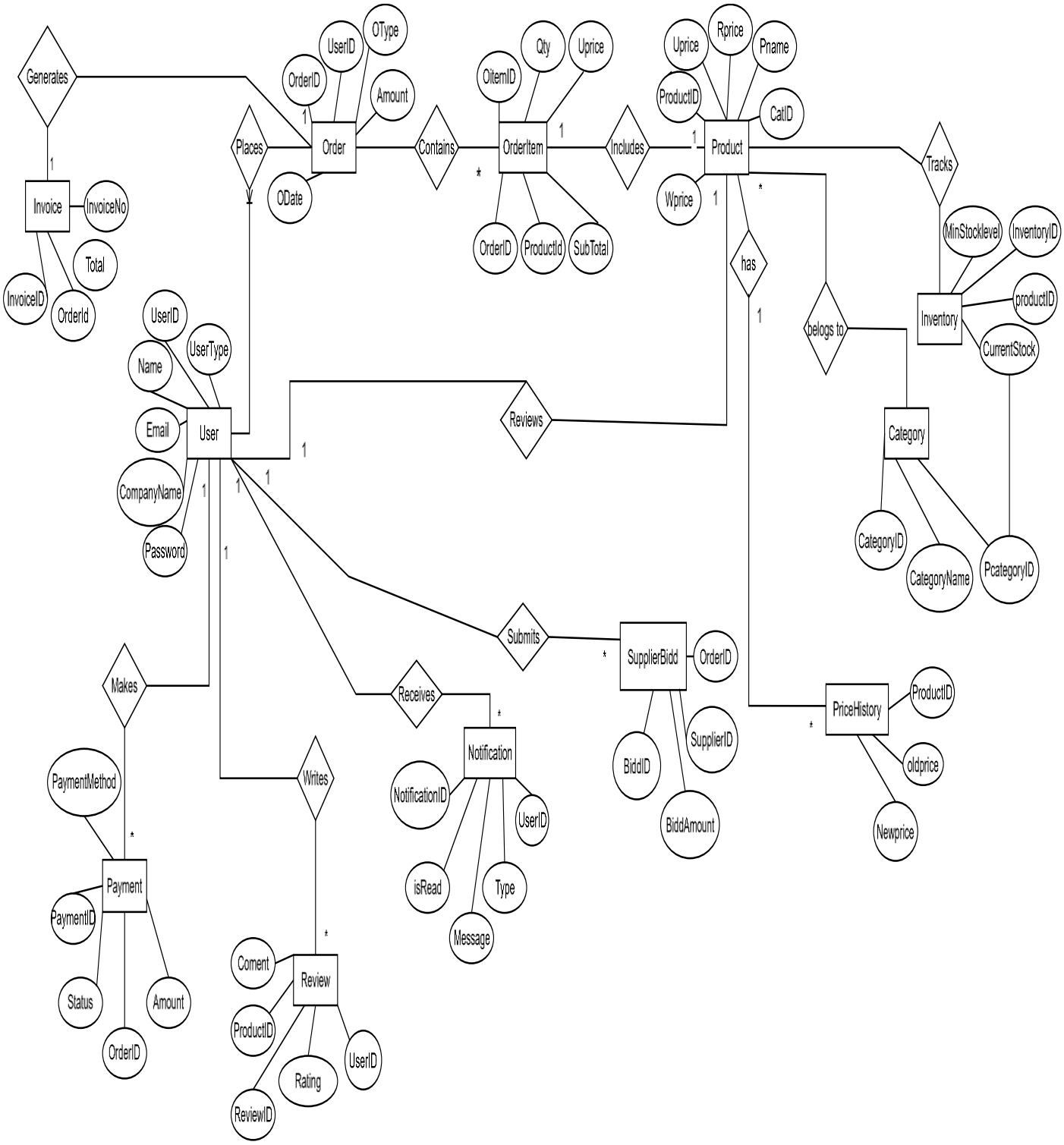
- The system provides feedback to users through:
- **Email Notifications:** Automatic email notifications via Nodemailer for order status, bid results and price changes.
- **Dashboard:** Each user role has a personalised dashboard showing relevant orders, products and notifications.
- **Invoice Download:** Buyers can download a digital PDF invoice after a successful payment.

Expected Features

- **Secure Registration and Login:** All users register and log in with role-based access control.
- **Product Listing and Search:** Buyers can search and filter products across all verified suppliers.
- **Order Placement and Payment:** Buyers can place orders and pay securely through Stripe.
- **Bidding System:** Sellers can submit bulk order requests and compare supplier bids.
- **User Dashboard:** All roles have access to a personalised dashboard with role-specific features.

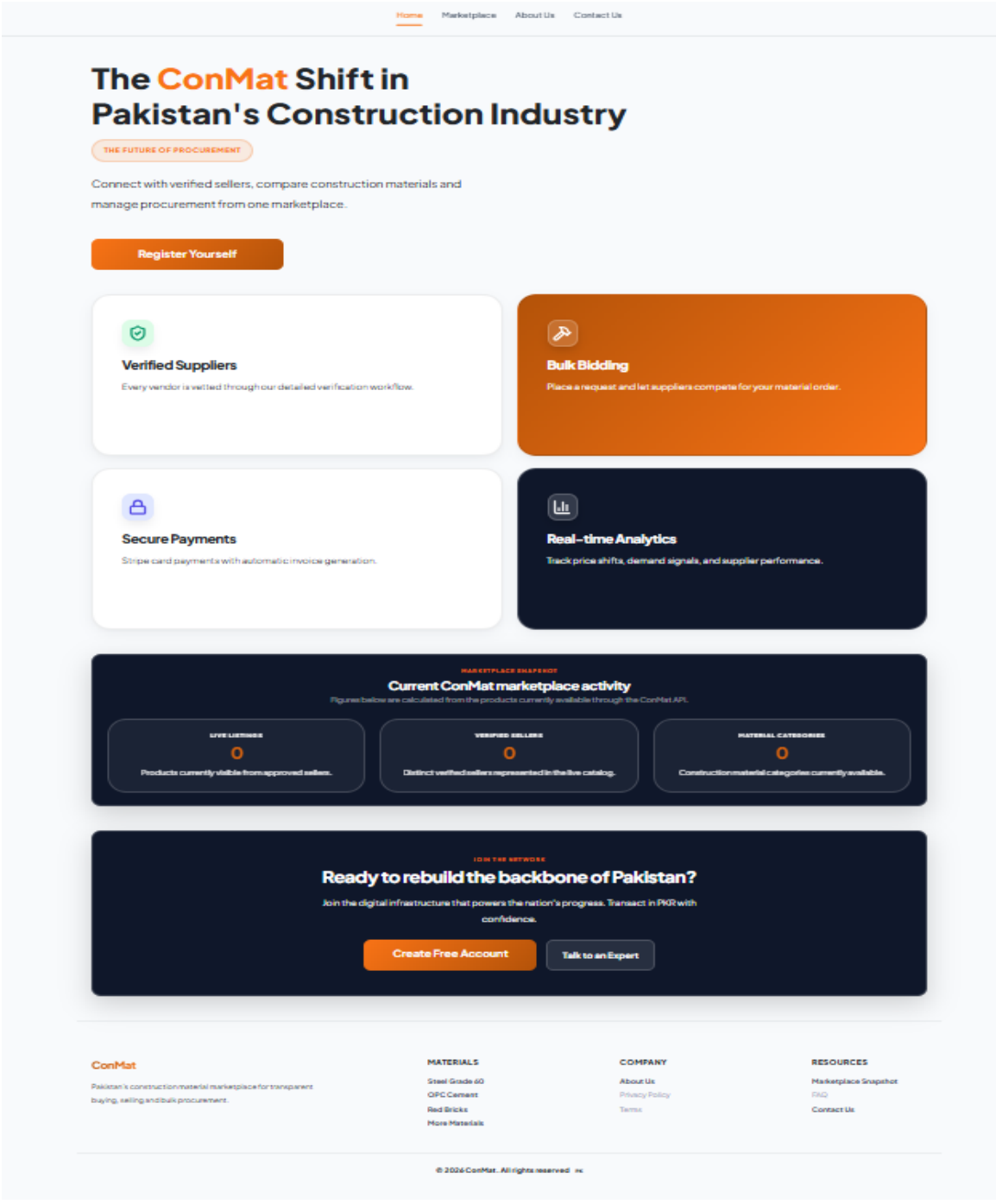
ER Diagram:

Figure 3 - 10: ER Diagram

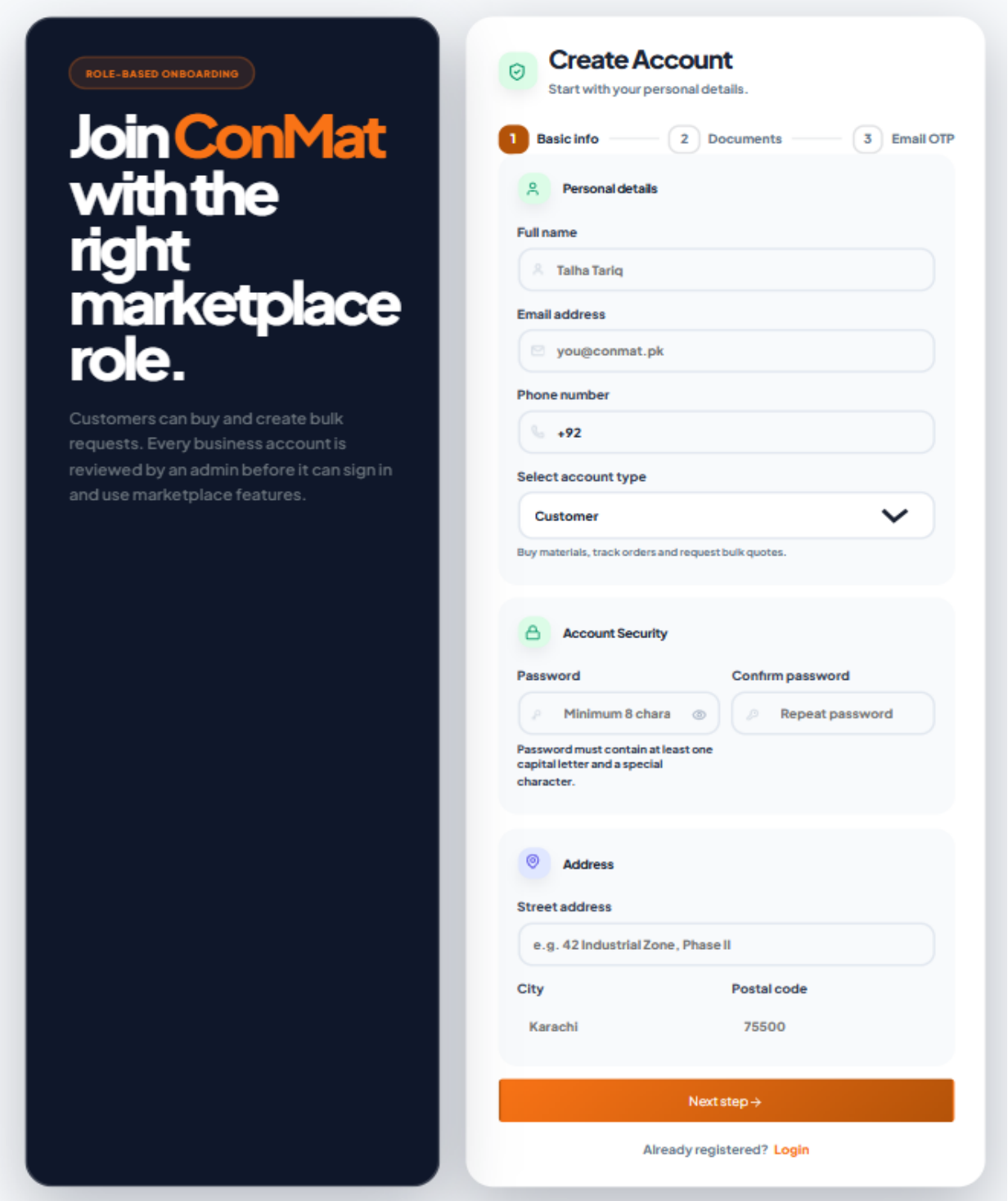


3.5.1 Screen Images

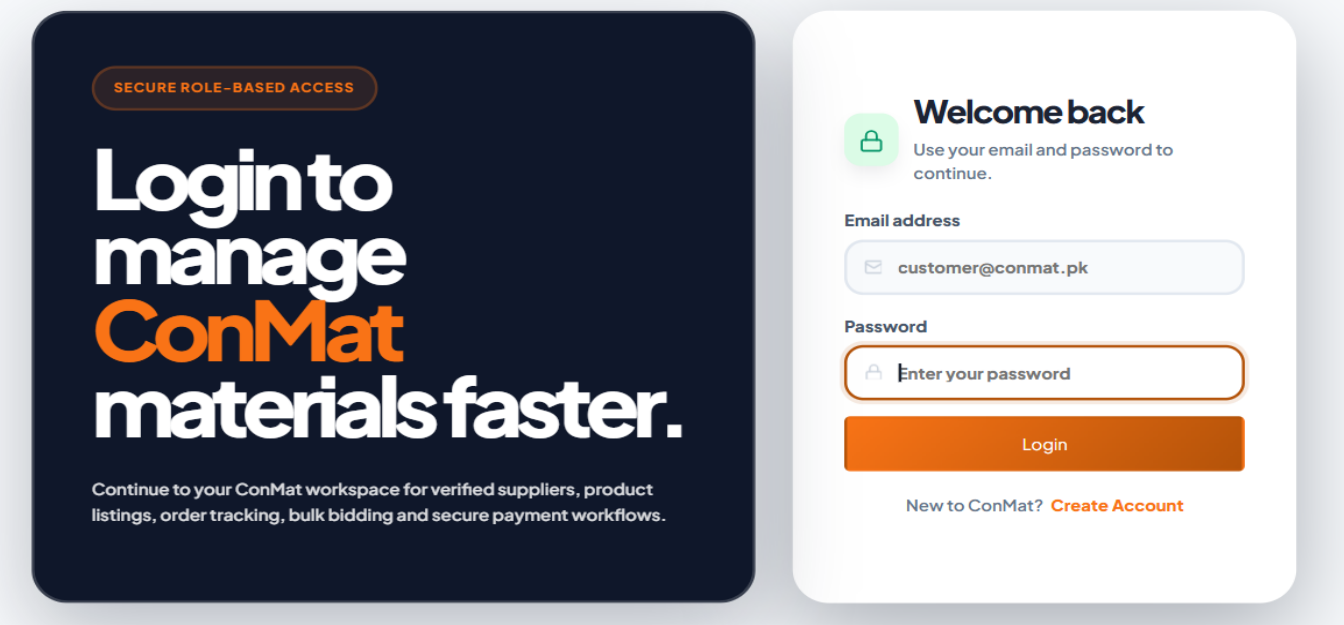
Landing Page:



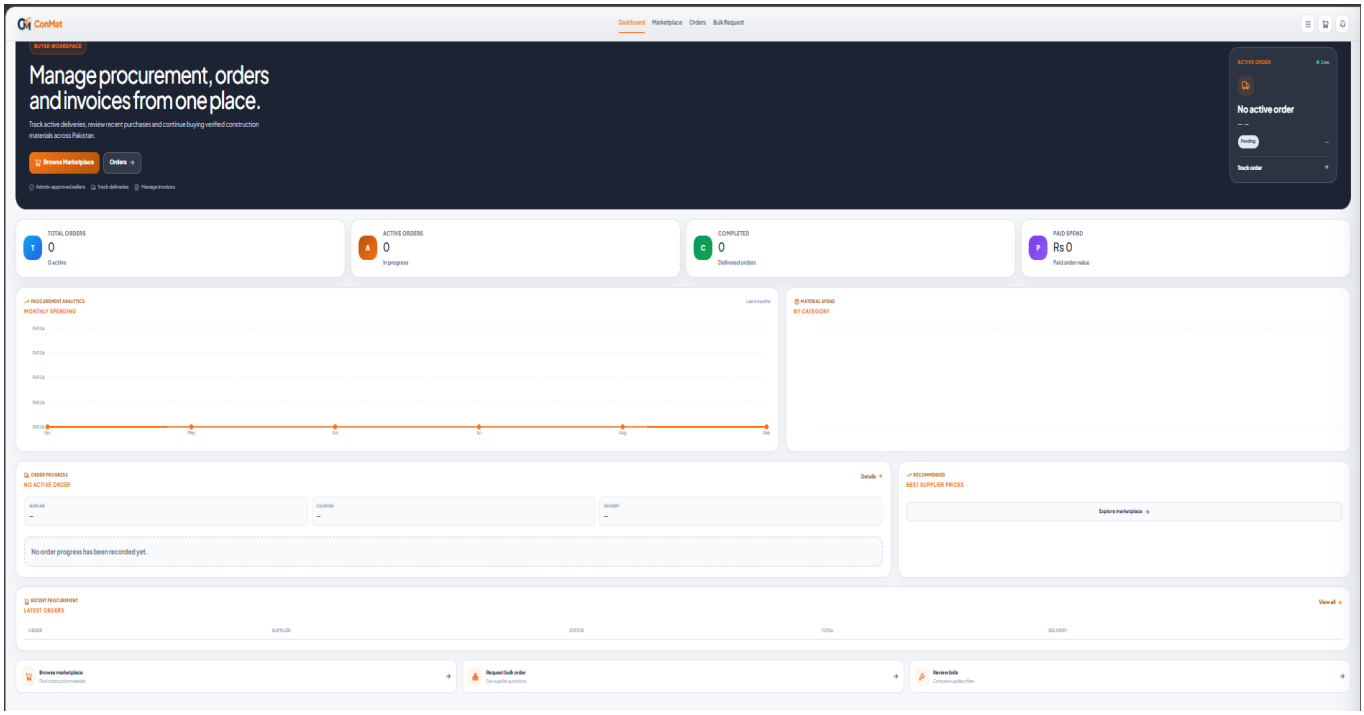
Registration Page:



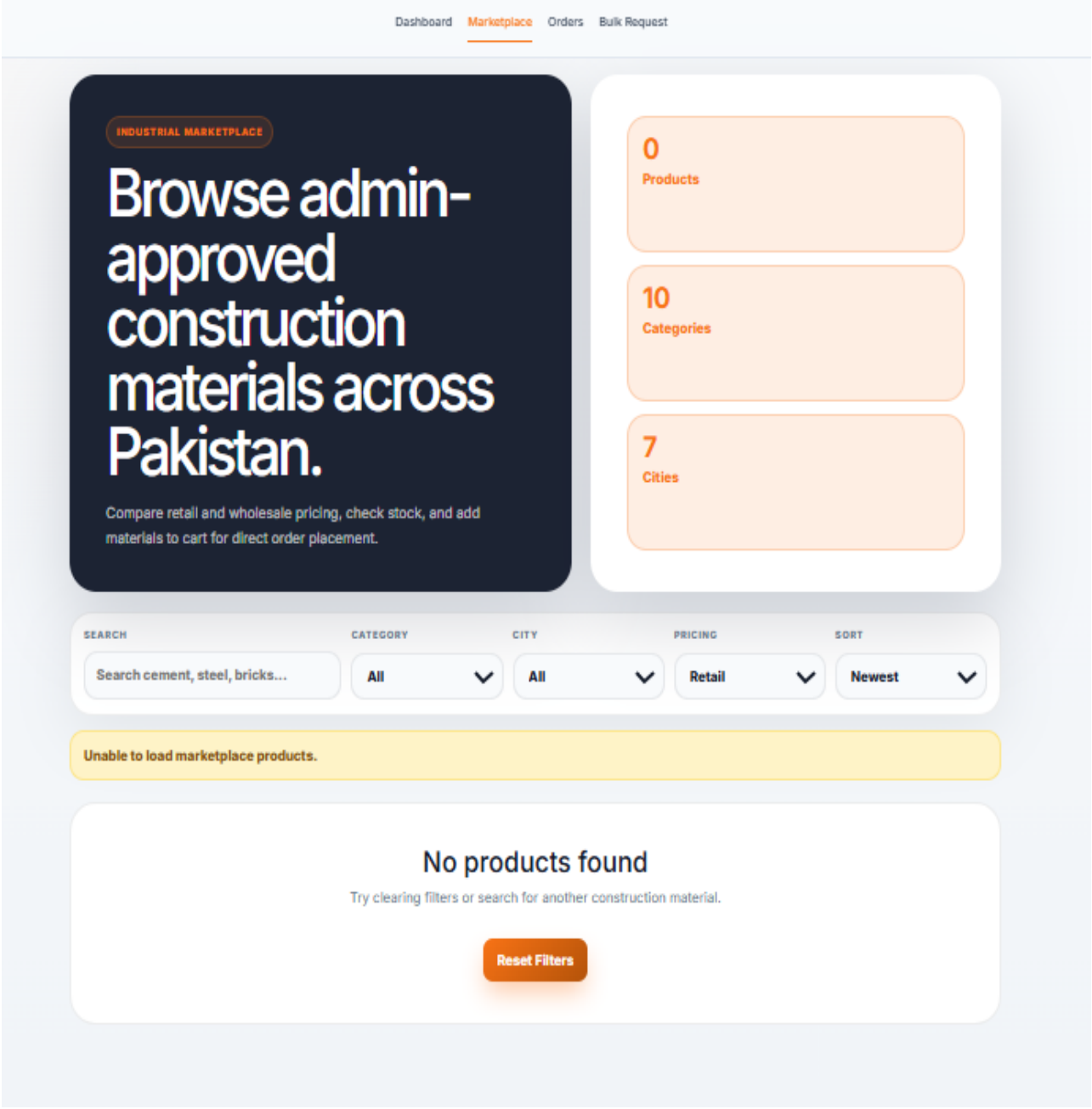
Login page:



Customer Dashboard:



Marketplace Dashboard:



3.6 Design Decisions

Here is how we can explain the design choices while developing the Construction Material platform:

Design Choice

1. Front-End Framework

React.Js: This enables dynamic and reusable UI components that reflect application state. We chose React.js as the front-end framework for building the user interface across all five user role dashboards.

2. Back-End Framework

Node.js with Express.js: This handles the logic of request and response. It bridges Models and Views. Node.js provides an event-driven, non-blocking I/O model, while Express.js simplifies backend routes, APIs and server handling.

3. Database Management System

PostgreSQL with Sequelize: This provides structured data modeling and easy database interactions. PostgreSQL was chosen because the platform is a marketplace that requires multiple databases different database for each service (users, products, orders, payments, bids). PostgreSQL supports multiple schemas and databases natively, making it suitable for this architecture. Sequelize ORM simplifies queries and model relationships.

4. Authentication and Authorization

JWT (Stateless Authentication): JWT tokens are managed by auth controller for secure session. This ensures only authorized users can access role-specific features.

5. Payment Gateway

Stripe: Stripe is used for online payment integration. It provides PCI-compliant secure transaction management.

6. RESTful API Design

RESTful API: Controllers reveal REST endpoints for frontend-backend communication. The API follows standard HTTP methods (GET, POST, PUT, DELETE).

7. MVC Architecture

MVC Architecture: Chosen for clear separation of concerns, maintainability and scalability in web applications. Models, Views and Controllers are organized into separate modules and folders.

3.7 Summary

The system is built on an object-oriented design pattern to ensure modularity, reusability, and scalability. It adopts the MVC architecture, providing a clear separation of concerns that enhances maintainability and scalability in web applications. RESTful API design is implemented with controllers exposing clean endpoints for seamless frontend-backend communication. Robust error handling mechanisms are integrated to maintain system stability and deliver meaningful error messages to users. The modular structure facilitates easier maintenance and updates, while scalability considerations allow for future enhancements such as logistics tracking. Security is prioritized through JWT authentication, Bcrypt password hashing and Stripe payment integration, safeguarding sensitive data. Usability is achieved by offering a clean, responsive and role-specific interface tailored to different user types. Finally, the maintainable codebase, organized with MVC and modular principles, ensures that the system remains easy to update and extend overtime.

Chapter 4:

Implementation

4. Implementation

This chapter describes the implementation details of the Construction Material platform. The backend is built with Node.js and Express.js, the frontend with React.js, and the database layer uses PostgreSQL accessed through Sequelize ORM. Before starting, make sure Node.js and PostgreSQL are installed. Create a PostgreSQL database and update the database configuration in the .env file with the correct host, port, database name, username and password. Install all dependencies by running Npm install in both the backend and frontend directories. Run database migrations using the Sequelize CLI to create all required tables. Start the backend server using node server.js or Npm start and start the React frontend using Npm start. The Stripe API keys and Nodemailer SMTP credentials must also be configured in the .env file before starting the server.

This chapter provides the major logic algorithms for all core modules to help understand the complete flow of the platform.

4.1 Algorithms

Table 4 - 1: Algorithm 1

Name	**Algorithm 1: User Registration Component**
Input	name, email, phone, password, role
Output	New user account created in PostgreSQL Users DB, redirect to Login page

Steps	1. Initialize registration form and input validators. 2. Collect user inputs: name, email, phone, password and role selection. 3. Validate all fields (non-empty, valid email format, password strength). 4. If validation fails, show inline error messages and stop. 5. Hash password using Bcrypt.js. 6. Check if email already exists in Users DB. 7. If email exists, show error: 'Email already registered.' 8. Insert new user record into Users DB with hashed password and role. 9. If role is Supplier, set account status to Pending Verification. 10. Redirect user to Login page with success message.

Table 4 - 2: Algorithm 2

Name	**Algorithm 2: User Login and JWT Authentication**
Input	email, password
Output	JWT token issued, user redirected to role-specific dashboard

Steps	1. Initialize login form and authentication middleware. 2. Collect user inputs: email and password. 3. Validate inputs (non-empty, valid email format). 4. Query Users DB for record matching the provided email. 5. If no user found, return error: 'Invalid credentials.' 6. Compare provided password against stored Bcrypt.js hash. 7. If password does not match, return error: 'Invalid credentials.' 8. If user role is Supplier and status is Pending, return error: 9. 'Account awaiting Admin verification.' 10. Generate JWT token with userId, email and role as payload. 11. Return JWT token to client. 12. Client stores token and redirects user to role-specific dashboard.

Table 4 - 3: Algorithm 3

Name	**Algorithm 3: Seller Verification by Admin**
Input	sellerId, action (approve / reject), adminToken
Output	Seller account status updated in Users DB, email notification sent
Steps	1. Admin opens Seller Verification section on Admin dashboard. 2. System queries Users DB for all Supplier accounts with status Pending. 3. Display list of pending suppliers with registration details. 4. Admin selects a seller and clicks Approve or Reject. 5. Validate adminToken to confirm the request is from an authenticated Admin.
	6. If action is Approve: update seller status to Approved in Users DB. 7. If action is Reject: update seller status to Rejected in Users DB. 8. Send email notification to seller via Nodemailer with approval or rejection result. 9. Update Admin dashboard to reflect the updated supplier status.

Table 4 - 4: Algorithm 4

Name	Algorithm 4: Add Product Listing Component
Input	Name, description, category, price, minWholesaleQty, stock, image file, sellerToken
Output	New product saved in Products DB and visible on marketplace
Steps	1. Seller opens Add Product form on Seller dashboard. 2. Collect product details: name, description, category, price, minWholesaleQty and stock. 3. Validate all fields (non-empty, price and stock must be positive numbers). 4. If validation fails, show error messages and stop. 5. Upload product image file via Multer to server storage. 6. If image upload fails, return error: 'Image upload failed. Check file size restrictions.' 7. Extract sellerId from JWT token in request header. 8. Verify seller status is Approved in Users DB. 9. Insert new product record into Products DB linked to sellerId. 10. Return success response. Product is now visible on the marketplace.

Table 4 - 5: Algorithm 5

Name	**Algorithm 5: Product Search and Filter Component**
Input	searchQuery, category (optional), minPrice (optional), maxPrice (optional)
Output	Filtered list of products from Products DB displayed to buyer
Steps	1. Buyer opens the Marketplace page. 2. System loads all verified supplier product listings from Products DB. 3. Buyer enters a search term or selects a category filter.

	4. Apply search filter: match products where name or description contains searchQuery. 5. Apply category filter if selected. 6. Apply price range filter if minPrice or maxPrice provided. 7. If no results match, display 'No products found' message. 8. Return filtered product list and render product cards. 9. On product card click, navigate to product detail page.

Table 4 - 6: Algorithm 6

Name	**Algorithm 6: Place Order Component**
Input	productId, quantity, shippingAddress, buyerToken
Output	Order created in Orders DB, stock reserved, supplier notified
Steps	1. Buyer selects a product and enters quantity and shipping address. 2. Validate inputs (quantity must be a positive integer, shipping address must not be empty). 3. Extract buyerId from JWT token. 4. Query Products DB to check current stock level for productId. 5. If stock is insufficient, return error: 'Insufficient stock available.' 6. Create order record in Orders DB with status Created. 7. Reserve stock by reducing available quantity in Products DB. 8. Query Orders DB to confirm order creation. 9. Send email notification to supplier via Nodemailer: 'New order received.' 10. Return order confirmation to buyer with orderId.

Table 4 - 7: Algorithm 7

Name	**Algorithm 7: Payment Processing and Invoice Generation**
Input	orderId, paymentMethod, cardDetails, buyerToken
Output	Payment recorded in Payments DB, invoice generated, order status updated to Paid
Steps	1. Buyer clicks Proceed to Payment on the order summary page. 2. Extract buyerId and verify JWT token. 3. Query Orders DB to fetch order details for orderId. 4. Validate order status is Pending before proceeding. 5. Send payment request to Stripe API with order amount and card details. 6. Await Stripe response. 7. If Stripe returns failure: return error 'Payment failed. Please try again.' Order status remains Pending. 8. If Stripe confirms payment: record transaction in Payments DB with transactionId, method and paidAt. 9. Update order status to Paid in Orders DB. 10. Generate PDF invoice containing order summary and payment details. 11. Store invoice PDF URL in Payments DB. 12. Send order confirmation email with invoice link to buyer via Nodemailer. 13. Return invoice download link to buyer.

Table 4 - 8: Algorithm 8

Name	**Algorithm 8: Bidding Module Component**
Input	productId, requiredQuantity, notes, sellerToken (for request); orderId, bidAmount, supplierToken (for bid submission)
Output	bulk order request saved in Bids DB, bids saved, accepted bid triggers order creation
Steps	1. Seller opens Request Bulk Order form. 2. Collect inputs: productId, requiredQuantity and optional notes. 3. Extract wholesalerId from JWT token. 4. Save bulk order request in Bids DB with status Open. 5. Send email notification to all verified suppliers via Nodemailer. 6. Supplier opens Bidding section on Supplier dashboard. 7. System loads all Open bulk order requests from Bids DB. 8. Supplier selects a request and enters bidAmount. 9. Extract supplierId from JWT token and verify Approved status. 10. Save bid in Bids DB linked to the bulk order request. 11. Wholesaler opens My Bids section and views all bids for their request. 12. Wholesaler selects a bid and clicks Accept. 13. System creates an order record from the accepted bid in Orders DB. 14. Update bid status to Accepted. Update other bids to Rejected. 15. Send email notification to winning supplier via Nodemailer.

Table 4 - 9: Algorithm 9

Name	**Algorithm 9: Admin Report Generation Component**
Input	adminToken, reportType (orders / revenue / users), dateRange (optional)
Output	Report data fetched from PostgreSQL databases and displayed on Admin dashboard

Steps	1. Admin opens the Reports section of the Admin dashboard. 2. Validate adminToken and confirm role is Admin. 3. Accept report parameters: reportType and optional dateRange. 4. If reportType is orders: query Orders DB for total orders grouped by status. 5. If reportType is revenue: query Payments DB for total revenue within dateRange. 6. If reportType is users: query Users DB for registration counts grouped by role.
	7. Aggregate results and structure report data. 8. If no data exists, return 'No data to display' message. 9. Return report data and render on Admin dashboard.

4.2 External APIs / SDKs

The following third-party APIs and SDKs are used in the implementation of the Construction Material platform:

Name & Version	Description	Purpose of Usage	Endpoint / Function / Class
Stripe v18.5.0	A PCI-compliant payment processing SDK. Handles secure online payment transactions.	Used to process buyer payments for orders. Handles card validation, charge creation and transaction confirmation.	paymentController.js stripe.paymentIntents.create() stripe.paymentIntents.confirm()
Nodemailer v6.x	An SMTP-based email sending library for Node.js. Supports HTML email templates and attachments.	Used to send automated email notifications for order confirmations, supplier alerts, bid results and price updates.	emailService.js transporter.sendMail() notificationController.js

Multer v2.0.2	A middleware for handling multipart/form-data, used for file uploads in Node.js.	Used to handle product image uploads from the Supplier dashboard. Validates file size and type before saving to server storage.	uploadMiddleware.js multer.diskStorage() productController.js

JWT (jsonwebtoken v9.0)	A library for generating and verifying JSON Web Tokens for stateless authentication.	Used to issue JWT tokens on login and verify them on all protected API routes for role-based access control.	authController.js jwt.sign() jwt.verify() authMiddleware.js
Sequelize v6.x	A Promise-based ORM for Node.js that supports PostgreSQL. Simplifies query building and model relationships.	Used for all PostgreSQL database interactions including model definitions, migrations, associations and queries.	models/ (User, Product, Order, Payment, Invoice, Bid) sequelize.sync() Model.findAll()
Axios v1.6.x	A Promise-based HTTP client for JavaScript used for client-side API communication.	Used in the React.js frontend to make all HTTP requests to the Node.js backend REST API.	services/api.js axios.get() axios.post() axios.put()

Bcrypt.js v5.1.x	A library for hashing and comparing passwords using the bcrypt algorithm.	Used to hash all user passwords before storing them in the Users DB and to compare passwords during login.	authController.js bcrypt.hash() bcrypt.compare()

4.3 Code Repository

Git Repository Link: **https://github.com/sh3erii/ConMat-FYP**

It is a public repository; entire code is there and will be updated on daily basis as We test our web-app and improve it..

4.4 Summary

This chapter outlined the implementation details of the Construction Material platform. Nine core algorithms were documented covering all major modules: user registration and login, supplier verification, product listing, product search, order placement, payment processing and invoice generation, bidding and admin report generation. The third-party APIs and SDKs used Stripe, Nodemailer, Multer, JWT, Sequelize, Axios and Bcrypt.js were described along with their purpose and integration points. The Git repository structure and contribution metrics were also outlined. These implementation details provide a clear understanding of the platform's working flow and how each module functions within the system.

Chapter 5:

System Testing

5. Introduction

This chapter focuses on testing and evaluation of the Construction Material platform. Software testing is conducted to ensure verification and validation of all system components. Testing is performed manually using Postman for API testing and browser-based end-to-end verification. Test cases are organized into Unit Testing, Functional Testing and Integration Testing categories. Each test case is documented with a unique identifier, preconditions, test steps, expected results and actual results.

5.1 Unit Testing (UT)

5.1.1 Test Case 1

Table 5 - 1: Unit Test 1

Field	**Details**
Testcase ID	UT1
Requirement ID	FR-1
Title	User Registration, Password Hashing
Description	This test verifies that user passwords are correctly hashed using Bcrypt.js before being stored in the PostgreSQL Users DB.
Objective	To confirm that the registration function hashes the password and does not store plain text passwords in the database.
Driver/Precondition	PostgreSQL database is running. Users DB table exists. Bcrypt.js is installed and configured.

Test Steps	1. Send a POST request to /api/auth/register with valid name, email, password and role. 2. Query the Users DB to retrieve the stored password_hash for the registered email. 3. Verify that the stored value is a Bcrypt hash and not the original plain text password.
Input	{ name: 'Test User', email: 'test@example.com', password: 'Test@1234', role: 'Customer' }
Expected Results	User record created in Users DB. password_hash field contains a Bcrypthashed string, not plain text.
Actual Result	password_hash stored correctly as Bcrypt hash. Plain text password not stored.
Remarks	Pass

5.1.2 Test Case 2

Table 5 - 2: Unit Test Case 2

Field	**Details**
Testcase ID	UT2
Requirement ID	FR-2
Title	JWT Token Generation on Login

Description	This test verifies that a valid JWT token is generated and returned when a registered user logs in with correct credentials.
Objective	To confirm that the login function generates a JWT token containing the correct userId, email and role fields.
Driver/Precondition	A registered user account exists in the Users DB. JWT secret is configured in .env file.
Test Steps	1. Send a POST request to /api/auth/login with valid email and password. 2. Inspect the response body for a token field. 3. Decode the JWT token and verify payload contains userId, email and role.
Input	{ email: 'test@example.com', password: 'Test@1234' }
Expected Results	Response returns HTTP 200 with a JWT token. Decoded token contains correct userId, email and role.
Actual Result	JWT token returned successfully with correct payload fields.
Remarks	Pass

5.1.3 Test Case 3

Table 5 - 3: Unit Test Case 3

Field	**Details**

Testcase ID	UT3
Requirement ID	FR-4
Title	Product Save to Products DB
Description	This test verifies that a seller can successfully add a product and the record is saved correctly in the Products DB.
Objective	To confirm that the addProduct function correctly inserts a product record into the Products DB with all required fields.
Driver/Precondition	An Admin-verified seller account exists. Seller is authenticated with a valid JWT token. Products DB table exists.
Test Steps	1. Send a POST request to /api/products/add with product data and seller JWT in Authorization header. 2. Query the Products DB to retrieve the newly inserted product record. 3. Verify all fields match the submitted data.
Input	{ name: 'OPC Cement 50kg', category: 'Cement', price: 1200, stock: 500, minWholesaleQty: 50 }
Expected Results	Product record inserted in Products DB with all fields matching input. Product is linked to the correct sellerId.

Actual Result	Product saved correctly in Products DB with correct sellerId and all fields.
Remarks	Pass

5.1.4 Test Case 4

Table 5 - 4: Unit Test Case 4

Field	**Details**
Testcase ID	UT4
Requirement ID	FR-6
Title	Order Creation and Stock Reservation
Description	This test verifies that placing an order correctly creates an order record in the Orders DB and reduces the product stock in the Products DB.
Objective	To confirm that the placeOrder function creates the order record and reserves stock atomically.
Driver/Precondition	A product with sufficient stock exists in Products DB. A buyer account is authenticated with a valid JWT token.

Test Steps	1. Send a POST request to /api/orders/place with productId, quantity and shippingAddress. 2. Query the Orders DB to confirm the order record exists with status Created. 3. Query the Products DB to confirm stock has been reduced by the ordered quantity.
Input	{ productId: 1, quantity: 10, shippingAddress: 'Gujranwala, Pakistan' }
Expected Results	Order record created in Orders DB with status Created. Product stock reduced by 10 in Products DB.
Actual Result	Order created and stock updated correctly.
Remarks	Pass

5.2 Functional Testing (FT)

5.2.1 Test Case 1

Table 5 - 5: Functional Test Case 1

Field	**Details**
Testcase ID	FT1
Requirement ID	FR-1
Title	User Registration with Role Selection

Description	Validate that a new user can register by providing all required details and selecting a role. Verify that the account is created and accessible.
Objective	Ensure that a user can successfully register on the Construction Material platform with all roles.
Driver/Precondition	Internet connection is active. PostgreSQL database is running.
Test Steps	1. Open the platform registration page. 2. Enter name, email, phone, password and select role (Customer). 3. Click Register. 4. Verify redirect to login page with success message.
Input	name: Ali Hassan, email: ali@test.com, phone: 03001234567, password: Ali@1234, role: Customer
Expected Results	User registered successfully. Redirect to login page. Record created in Users DB.
Actual Result	Registered successfully. User record created in PostgreSQL Users DB.
Remarks	Pass

5.2.2 Test Case 2

Table 5 - 6: Functional Test Case 2

Field	**Details**
Testcase ID	FT2
Requirement ID	FR-3
Title	Admin Approves Seller Account

Description	Verify that the Admin can view pending seller accounts and approve them so their products become visible on the marketplace.
Objective	Ensure the seller verification workflow functions correctly end-to-end.
Driver/Precondition	A supplier account with Pending status exists. Admin is logged in.
Test Steps	1. Admin logs in to Admin dashboard. 2. Admin navigates to Seller Verification. 3. Admin selects a pending supplier and clicks Approve. 4. Verify seller status is updated to Approved.
Input	sellerId: 5, action: Approve
Expected Results	Seller status updated to Approved in Users DB. Seller can now add product listings.
Actual Result	Seller approved successfully. Status updated in database.
Remarks	Pass

5.2.3 Test Case 3

Table 5 - 7: Functional Test Case 3

Field	**Details**
Testcase ID	FT3
Requirement ID	FR-5
Title	Product Search with Category Filter

Description	Verify that buyers can search and filter products on the marketplace and see only relevant results.
Objective	Ensure the product search and filter feature works correctly for all buyer roles.
Driver/Precondition	Verified seller products exist in the Products DB. Buyer is logged in.
Test Steps	1. Buyer opens the Marketplace page. 2. Buyer selects category: Cement from the filter dropdown. 3. Observe the product listing results. 4. Verify only Cement category products are shown.
Input	category: Cement
Expected Results	Only products with category Cement are displayed. Other categories are filtered out.
Actual Result	Products filtered correctly. Only Cement products displayed.
Remarks	Pass

5.2.4 Test Case 4

Table 5 - 8: Functional Test Case 4

Field	**Details**
Testcase ID	FT4
Requirement ID	FR-7
Title	Stripe Payment and Invoice Download

Description	Verify that a buyer can complete a payment through Stripe and download the generated invoice after successful payment.
Objective	Ensure the end-to-end payment and invoice generation flow works correctly.
Driver/Precondition	A placed order with status Pending exists. Stripe is configured. Buyer is authenticated.
Test Steps	1. Buyer opens Order Summary and clicks Proceed to Payment. 2. Enter Stripe test card details. 3. Submit payment. 4. Verify order status changes to Paid. 5. Click Download Invoice button. 6. Verify PDF invoice is downloaded.
Input	Stripe test card: 4242 4242 4242 4242, exp: 12/26, CVC: 123
Expected Results	Payment processed. Order status updated to Paid. PDF invoice generated and available for download.
Actual Result	Payment processed via Stripe. Invoice generated and downloaded successfully.
Remarks	Pass

5.2.5 Test Case 5

Table 5 - 9: Functional Test Case 5

Field	Details

Testcase ID	FT5
Requirement ID	FR-8
Title	Verified seller Submits Bulk Order Request and Accepts Bid
Description	Verify that a seller can submit a bulk order request, suppliers can submit bids and the seller can accept the best bid to initiate an order.
Objective	Ensure the full bidding workflow functions correctly end-to-end.
Driver/Precondition	Seller and at least one verified supplier are authenticated. Product exists in Products DB.
Test Steps	1. Seller opens Bidding page and submits a bulk order request with productId and quantity. 2. Supplier logs in and submits a bid amount for the request. 3. Selller views all bids and clicks Accept on the best bid. 4. Verify an order is created in Orders DB from the accepted bid.
Input	productId: 3, requiredQuantity: 500, bidAmount: 580000
Expected Results	Bulk order request saved. Supplier bid saved. Accepted bid creates an order in Orders DB. Winning supplier notified via email.
Actual Result	Bidding workflow completed. Order created from accepted bid. Email notification sent.
Remarks	Pass

5.3 Integration Testing (IT)

5.3.1 Test Case 1

Table 5 - 10: Integration Test Case 1

Field	Details
Testcase ID	IT1
Requirement ID	FR-6, FR-7
Title	Order Placement Payment Invoice Generation Integration
Description	Tests the end-to-end integration between the Order Management module, Payment Processing module and Invoice Generation. Verifies that placing an order and completing payment correctly triggers invoice creation.
Objective	Ensure data flows correctly from order creation through Stripe payment to invoice generation and storage in Payments DB.
Driver/Precondition	Buyer is authenticated. Product with sufficient stock exists. Stripe is configured. PostgreSQL databases are running.

Test Steps	1. Buyer places an order via POST /api/orders/place. 2. Verify order created in Orders DB with status Created. 3. Buyer completes payment via POST /api/payments/process. 4. Verify payment recorded in Payments DB. 5. Verify order status updated to Paid in Orders DB. 6. Verify invoice PDF generated and URL stored in Payments DB.
Input	{ productId: 2, quantity: 5, shippingAddress: 'Lahore, Pakistan' }, Stripe test card
Expected Results	Order → Payment → Invoice chain completes successfully. All records updated in respective PostgreSQL databases.
Actual Result	Data flowed correctly through all three modules. Records updated in Orders DB and Payments DB.
Remarks	Pass

5.3.2 Test Case 2

Table 5 - 11: Integration Test Case 2

Field	**Details**
Testcase ID	IT2
Requirement ID	FR-4, FR-5

Title	Seller Adds Product Product Appears on Marketplace
Description	Tests the integration between the Supplier Product Management module and the Marketplace browsing module. Verifies that a newly added product is immediately retrievable by buyers on the marketplace.
Objective	Validate the end-to-end data flow from product creation through to marketplace display.
Driver/Precondition	Admin-verified seller is authenticated. Products DB is running.
Test Steps	1. Supplier sends POST /api/products/add with valid product data. 2. Buyer sends GET /api/products with category filter matching new product. 3. Verify the newly added product appears in the response.
Input	{ name: 'Steel Rod 12mm', category: 'Steel', price: 4500, stock: 200, minWholesaleQty: 20 }
Expected Results	Product saved in Products DB. Product appears in buyer's marketplace results for category Steel.
Actual Result	Product added and immediately visible in marketplace search results.
Remarks	Pass

5.3.3 Test Case 3

Table 5 - 12: Integration Test Case 3

Field	Details
Testcase ID	IT3
Requirement ID	FR-8, FR-9
Title	Bulk Order Request Supplier Bid Email Notification Integration
Description	Tests the integration between the Bidding module and the Email Notification module. Verifies that submitting a bulk order request triggers email notifications to relevant suppliers via Nodemailer.
Objective	Ensure data flows correctly from bid request creation through Nodemailer email dispatch.
Driver/Precondition	Seller and verified supplier are authenticated. Nodemailer SMTP is configured. Bids DB is running.
Test Steps	1. Seller sends POST /api/bids/request with productId and quantity. 2. Verify bulk order request saved in Bids DB with status Open. 3. Verify email notification sent to verified suppliers via Nodemailer.
Input	{ productId: 1, requiredQuantity: 300, notes: 'Required within 2 weeks' }
Expected Results	Bulk request saved in Bids DB. Email notification dispatched to relevant suppliers by Nodemailer.

Actual Result	Request saved. Email notification sent successfully to supplier email addresses.
Remarks	Pass

5.3.4 Test Case 4

Table 5 - 13: Integration Test Case 4

Field	**Details**
Testcase ID	IT4
Requirement ID	FR-3, FR-4
Title	Seller Verification Product Listing Activation Integration
Description	Tests the integration between the Admin Verification module and the Product Management module. Verifies that a supplier's products only become visible after Admin approval.
Objective	Ensure that product visibility on the marketplace is correctly controlled by supplier verification status.
Driver/Precondition	A seller with Pending status has added products. Admin is authenticated.

Test Steps	1. Seller adds a product while in Pending status. 2. Buyer queries GET /api/products product should NOT appear. 3. Admin approves the supplier via PATCH /api/admin/verify-seller. 4. Buyer queries GET /api/products again product should NOW appear.
Input	sellerId: 7, action: Approve
Expected Results	Product not visible before approval. Product visible after Admin approval.
Actual Result	Visibility correctly controlled by verification status.
Remarks	Pass

5.3.5 Test Case 5

Table 5 - 14: Integration Test Case 5

Field	**Details**
Testcase ID	IT5
Requirement ID	FR-6, FR-4
Title	Order Placement Inventory Stock Update Integration
Description	Tests the integration between the Order Management module and the Inventory module. Verifies that placing an order correctly reduces the product stock in the Products DB.

Objective	Ensure that inventory stock levels are accurately updated when an order is placed.
Driver/Precondition	Product with stock 100 exists in Products DB. Buyer is authenticated.
Test Steps	1. Record current stock level for productId: 4 (stock: 100). 2. Buyer places an order for quantity: 30. 3. Query Products DB for productid : 4 4. Verify stock reduced from 100 to 70.
Input	{ productId: 4, quantity: 30 }
Expected Results	Stock reduced from 100 to 70 in Products DB after order placement.
Actual Result	Stock correctly updated to 70.
Remarks	Pass

5.4 Summary

This chapter focused on testing and evaluation of the Construction Material platform. Unit testing, functional testing and integration testing were conducted across all core modules with satisfactory results. Unit tests verified individual functions including password hashing, JWT token generation, product saving and order creation. Functional tests validated end-to-end user workflows including registration, supplier verification, product search, Stripe payment, invoice download and the complete bidding lifecycle. Integration tests confirmed correct data flow between modules including the order-payment-invoice chain, supplier verification and marketplace visibility, bid request and email notification, and inventory stock updates. All test cases passed successfully.

Chapter 6:
System Conversion

6 Introduction

System conversion is the process of transitioning from one information system to another. It involves shifting data, processes and user operations from the development environment to the newly deployed production environment. Effective system conversion ensures minimal disruption, data integrity and seamless user adaptation. This chapter describes the conversion method selected for the Construction Material platform, the deployment steps, data conversion process, user training instructions, post deployment testing and challenges encountered during the process.

6.1 Conversion Method

For the deployment of the Construction Material platform, the selected conversion method is:

6.1.1 Pilot Conversion

In the pilot conversion approach, the new system is initially implemented in a limited, controlled environment with a selected group of users including suppliers, wholesalers and admin users. This allows comprehensive testing of the system's core functionalities, identification of real-world issues and collection of user feedback before full-scale public deployment.

Justification:

- **Reduced Risk:** Only a limited group of verified users interact with the new system initially, minimizing the risk of system-wide failures.
- **Focused Testing:** Allows in-depth testing of all five modules authentication, product management, order placement, payment processing and bidding before full rollout.
- **User Feedback Loop:** Early feedback from pilot users helps refine the platform's performance, usability and reliability before wider deployment.

Pilot Scope:

- **Initial Group:** Selected construction material suppliers, one wholesaler and admin users.
- **Modules Tested:** User Authentication, Product Listing, Order Placement, Stripe Payment, Invoice Generation and Bidding Module.
- **Duration:** Two weeks of controlled usage followed by full rollout after adjustments.

6.2 Deployment

Deployment involves preparing and transferring the Construction Material platform from the development environment to an operational environment accessible to all users. This includes installing dependencies, configuring PostgreSQL databases, setting up environment variables, running Sequelize migrations and starting the Node.js backend and React.js frontend servers.

6.2.1 Deployment Steps

Table 6 - 1: Deployment Steps

Step No.	Task	Details
1	**Environment Setup**	Install Node.js (v20.x) and PostgreSQL (v15.x) on the server. Configure system environment variables.
2	**Clone Repository**	Clone the project repository from Git using: git clone [repository-link]. Navigate to the project directory.
3	**Install Dependencies**	Run npm install in both the backend and frontend directories to install all required packages.
4	**Configure .env File**	Set all required environment variables in the .env file: PostgreSQL credentials, JWT secret, Stripe API keys, Nodemailer SMTP settings and Multer storage path.
5	**Database Setup**	Create PostgreSQL databases for each service domain (Users, Products, Orders, Payments, Bids). Run Sequelize migrations using: npx sequelize-cli db:migrate.
6	**Start Backend Server**	Start the Node.js and Express.js backend server using: node server.js or npm start. Verify all API routes are active.
7	**Build and Start Frontend**	Build the React.js frontend using: npm run build. Serve the build or start the development server using: npm start.

8	**Live Environment Verification**	Test all modules by sending requests through Postman. Verify database connectivity, Stripe payment integration and Nodemailer email delivery.
9	**Deployment Confirmation**	Conduct user acceptance testing with the pilot group. Confirm all core workflows function correctly before full rollout.

6.2.2 Data Conversion

Table 6 - 2: Data Conversion

Step	Task	Details
1	**Data Preparation**	Gather initial seed data including product categories, sample supplier accounts and admin credentials for populating the PostgreSQL databases.
2	**Data Cleaning**	Remove any inconsistencies, duplicates or formatting errors from the seed data before insertion.
3	**Data Transformation**	Format all seed data into the correct schema structure matching the Sequelize model definitions for each database.
4	**Data Loading**	Use Sequelize seeders (npx sequelize-cli db:seed:all) or custom scripts to populate the PostgreSQL databases with initial data.
5	**Data Validation**	Verify all inserted records are correct by querying each PostgreSQL database and checking field values against expected data.
6	**Backup Procedure**	Create a PostgreSQL database backup using pg_dump before going live. Store backups securely for recovery purposes.

6.2.3 Training

For training, step-by-step user instructions are provided for the two main use cases of the platform. These instructions serve as a user manual for the pilot group.

Table 6 - 3: Training Instruction 1

Step	Instruction
1	Open the Construction Material platform in a modern web browser.
2	Click Register and create an account by entering name, email, phone, password and selecting role as Customer, Retailer or Wholesaler.
3	Log in using your registered email and password.
4	Navigate to the Marketplace from the dashboard.
5	Use the search bar or category filters to find a construction material product.
6	Click on a product card to view details including price, stock and supplier.
7	Enter quantity and shipping address, then click Place Order.
8	Click Proceed to Payment and enter card details on the Stripe payment form.
9	Submit payment. On success, click Download Invoice to download your PDF invoice.

Table 6 - 4: Training Instruction 2

Step	Instruction
1	Register on the platform by selecting role as Supplier and wait for Admin approval.
2	Once approved, log in to your Supplier dashboard.
3	Navigate to Product Management from the sidebar.
4	Click Add Product and fill in product name, description, category, price, minimum wholesale quantity and stock level.

5	Upload a clear product image using the image upload button (Multer).
6	Click Save Product. Your product is now live on the marketplace.
7	To view and manage incoming orders, navigate to Order Management on your dashboard.

6.3 Post Deployment Testing

Post-deployment testing was conducted to validate that the Construction Material platform is correctly installed, configured and operating as intended. The objective was to ensure data integrity, feature functionality and user experience after deployment.

6.3.1 Post Deployment Testing

Table 6 - 5: Post Deployment Testing Steps

Step	Test Activity	Expected Outcome
1	Launch Application	Platform loads successfully in browser without errors or broken layouts.
2	User Registration and Login	All user roles can register and log in. JWT tokens are issued correctly. Role-specific dashboards load.
3	Supplier Verification	Admin can approve or reject supplier accounts. Approved supplier can add product listings.
4	Product Listing and Search	Verified seller products appear on the marketplace. Search and category filters return correct results.
5	Order Placement	Buyer can place an order. Order record created in Orders DB. Supplier receives email notification.
6	Stripe Payment	Payment processes correctly. Order status updates to Paid. PDF invoice generated and downloadable.
7	Bidding Module	Seller can submit bulk order request. Supplier can submit bid. Wholesaler can accept bid and order is created.

8	Email Notifications	Nodemailer sends correct email notifications for all trigger events: order placed, payment confirmed, bid received.
9	Data Integrity Verification	All records in PostgreSQL databases (Users, Products, Orders, Payments, Bids) persist correctly after operations.
10	Role-Based Access Control	Users can only access features assigned to their role. Unauthorized route access returns 401 or 403 response.

6.4 Challenges and Solutions

During deployment and post-deployment phases of the Construction Material platform, several technical and operational challenges were encountered. Below is a list of the main challenges along with their applied solutions:

Table 6 - 6: Challenges and Solutions

Challenge	Description	Solution Implemented
PostgreSQL Multiple Database Configuration	Configuring Sequelize to connect to multiple separate PostgreSQL databases for each service domain caused connection errors initially.	Created separate Sequelize instances in individual config files for each database and imported the correct instance in each model file.
Stripe Webhook Integration	Stripe payment confirmation webhooks were not reaching the local server during development, causing order status to not update automatically.	Used Stripe CLI to forward webhooks to the local server during development and configured a proper public endpoint for the production deployment.
Multer File Size Restrictions	Large product image uploads were being rejected by the server, causing supplier product creation to fail without a clear error message.	Added explicit file size limits and file type validation in the Multer configuration and returned a clear error message for oversized files.
JWT Token Expiry Handling	Users were silently logged out when JWT tokens expired without any notification, causing confusion on the frontend.	Added Axios interceptors on the frontend to detect 401 responses and redirect users to the login page with a session expired message.

CORS Configuration Issues	Cross-Origin Resource Sharing errors prevented the React.js frontend from communicating with the Node.js backend during deployment.	Configured the CORS middleware in Express.js with the correct allowed origins for both development and production environments.
Nodemailer SMTP Authentication Failure	Email notifications were failing silently due to incorrect SMTP credentials and Gmail security restrictions.	Used an App Password instead of a standard Gmail password and enabled the correct Gmail SMTP settings in the Nodemailer transporter configuration.
Sequelize Migration Conflicts	Running migrations on an already seeded database caused duplicate table errors and data inconsistencies.	Used Sequelize migration state tracking and added conditional checks in migration files to prevent duplicate table creation.

6.5 Summary

This chapter described the system conversion and deployment process for the Construction Material platform. The Pilot Conversion method was selected to minimize deployment risk and allow focused testing with a controlled user group before full rollout. The deployment steps covered environment setup, PostgreSQL database configuration, Sequelize migration, backend and frontend server startup, and live environment verification. Data conversion procedures including preparation, cleaning, transformation and loading were documented. User training instructions were provided for the two main use cases: placing an order as a buyer and adding a product as a supplier. Post-deployment testing confirmed correct functionality across all ten test activities. Seven deployment challenges were identified and resolved, covering PostgreSQL multi-database configuration, Stripe webhooks, Multer file restrictions, JWT expiry handling, CORS, Nodemailer SMTP and Sequelize migration conflicts.

Chapter 7:
Conclusion

7.1 Evaluation

This chapter will conclude what have been completed, partial or still under development.

Table 7 - 1: Progress Table

Objectives	Status
Add supplier, wholesaler, retailer and customer modules with role-based dashboards.	Completed
Implement user registration and login with JWT-based authentication and role-based access control.	Completed
Implement Admin verification and approval of supplier accounts before product listings go live.	Completed
Include product listing and inventory management with Multer image upload support.	Completed
Implement product browsing and search with category and price filters.	Completed
Implement order placement and tracking through the full lifecycle (Created, Pending, Paid, Completed, Cancelled).	Completed
Integrate secure Stripe payment processing.	Completed
Implement automatic digital PDF invoice generation on successful payment.	Completed
Provide bidding for bulk orders seller request, supplier bid, bid acceptance.	Completed
Integrate email notifications via Nodemailer for order status, bid results and price updates.	Completed
Implement Admin dashboard with user management, product management and report generation.	Completed
Build responsive UI using React.js and Bootstrap across all user role dashboards.	Completed

Configure PostgreSQL with multiple databases one per service domain using Sequelize ORM.	Completed
Encrypt all passwords using Bcrypt.js before storage.	Completed

7.2 Requirements Traceability Matrix

The following traceability matrix ensures that all functional requirements are accounted for by mapping each requirement to its design specification, implementation file and test case:

Table 7 - 2: Traceability Matrix

Req. ID	Requirement Description	Design Specification	Code	Test ID
FR-1	User Registration	authController.js / User Model	authRoutes.js	UT1, FT1
FR-2	User Login and Session Management	authController.js / JWT Middleware	authRoutes.js	UT2, FT1
FR-3	Seller Verification by Admin	adminController.js / User Model	adminRoutes.js	FT2, IT4
FR-4	Product Listing and Inventory Management	productController.js / Product Model	productRoutes.js	UT3, FT3, IT2
FR-5	Product Browsing and Search	productController.js / Product Model	productRoutes.js	FT3, IT2
FR-6	Order Placement and Tracking	orderController.js / Order Model	orderRoutes.js	UT4, FT5, IT1, IT5
FR-7	Payment Processing and Invoice Gen.	paymentController.js / Payment Model	paymentRoutes.js	FT4, IT1
FR-8	Bidding for Bulk Orders	bidController.js / Bid Model	bidRoutes.js	FT5, IT3

FR-9	Email Notifications	emailService.js / Nodemailer Config	notificationService.js	IT3
FR-10	Admin Panel	adminController.js / All Models	adminRoutes.js	FT2, IT4

7.3 Conclusion

The Construction Material Wholesale and Retail Marketplace Platform has been successfully developed and all fourteen specified objectives have been completed. The platform addresses the core problem of fragmented and manual construction material transactions in Pakistan by providing a centralized, digital solution for all stakeholders.

The system successfully connects verified suppliers, wholesalers, retailers and customers on a single webbased platform. Key achievements include a fully functional role-based authentication system using JWT, an Admin-controlled supplier verification workflow, dynamic product listings with real-time inventory management, Stripe-integrated secure payment processing, automated PDF invoice generation, a competitive static bidding system for bulk orders and Nodemailer-powered email notifications.

The platform is built on a robust technology stack React.js for the frontend, Node.js and Express.js for the backend and PostgreSQL with Sequelize ORM configured with separate databases per service domain as recommended by the viva conductor. All passwords are securely encrypted using Bcrypt.js and all session management is handled through stateless JWT tokens.

Objective	Functionality Provided
Role-Based Marketplace	Five role-specific dashboards: Admin, Supplier, Wholesaler, Retailer and Customer.
Seller Verification	Admin verifies and approves sellers before products go live, reducing fraud risk.
Product Management	Sellers list products with images, pricing and stock. Real-time inventory updates on order.

Order and Payment System	Full order lifecycle management with Stripeintegrated payments and PDF invoice generation.
Bidding for Bulk Orders	Wholesalers & retailers request bulk order quotes. Suppliers submit bids. Wholesaler & retailers accepts best offer.
Email Notifications	Automated Nodemailer emails for all key events: orders, payments, bids and price updates.
Admin Dashboard and Reports	Admin manages all users, products, orders and generates platform analytics reports.
PostgreSQL	Separate PostgreSQL databases for Users, Products, Orders, Payments and Bids via Sequelize ORM.

7.4 Future Work

After completion of the current project scope, the following enhancements and features are suggested for future development:

Real-Time Delivery Tracking: Integrate logistics and delivery tracking so buyers can monitor shipment status in real time. This was identified as outside the current scope but is a high-priority enhancement.

Mobile Application: Develop native Android and iOS mobile applications using React Native to extend platform accessibility beyond web browsers.

Advanced Analytics Dashboard: Expand the Admin reporting module with visual analytics including charts for sales trends, supplier performance, top-selling products and revenue forecasts.

AI-Powered Price Recommendation: Implement a machine learning model that recommends competitive prices for suppliers based on market trends and historical transaction data.

Buyer Rating and Review System: Allow buyers to rate and review verified suppliers after order completion to improve marketplace trust and accountability.

Microservices Architecture: Migrate the current monolithic Node.js backend to a microservices architecture to improve scalability, independent deployment and fault isolation for each service domain.

Two-Factor Authentication (2FA): Add optional two-factor authentication for supplier and admin accounts to further strengthen platform security.

References

- AliBaba.com. (2024, September 9). *Manufacturers, Suppliers, Exporters & Importers from the world's largest online B2B marketplace*. Retrieved from https://www.alibaba.com
- Construction Baazar. (2023, November 3). *Construction material in Pakistan | Buy & sell construction materials online*. Retrieved from https://constructionbaazar.com
- Home Point. (n.d.). *Building materials online*. Retrieved November 15, 2024, from https://homepoint.pk
- MDN Docs Web. (2025). *Web technology for developers*. Retrieved July 28, 2025, from https://developer.mozilla.org/en-US/docs/Web

- Node.js. (2025). *Node.js documentation*. Retrieved December 02, 2025, from https://nodejs.org
- OWASP Foundation. (2025). *OWASP Top Ten – The ten most critical web application security risks*. Retrieved September 10, 2025, from https://owasp.org
- PostgreSQL. (2026). *PostgreSQL documentation*. Retrieved May 1, 2026, from . https://www.postgresql.org/docs
- React.js. (2025). *React documentation*. Retrieved December 10, 2025, from https://react.dev
- Sequelize. (2025). *Sequelize ORM documentation*. Retrieved November 30, 2025, from https://sequelize.org
- Stripe API. (2025). *Stripe API documentation*. Retrieved December 1, 2025, from https://stripe.com/docs
- Zarea.pk. (2025). *Pakistan's biggest commodities B2B platform*. Retrieved October 10, 2025, from https://zarea.pk

Appendix A:

Use Case Description

UC-1: User Registration

Use Case ID	UC-1
Use Case Name	User Registration
Actors	Customer, Retailer, Wholesaler, Supplier
Description	Allows a new user to create an account by providing personal details, setting a password and selecting a role.
Trigger	User opens the platform and clicks Register on the landing page.
Preconditions	User must not already be registered with the same email address.
Postconditions	New user account is created in the PostgreSQL Users DB. Supplier accounts are marked Pending until Admin approval.
Normal Flow	1. User opens the platform. 2. User clicks Register. 3. User enters name, email, phone, password and selects role. 4. System validates all inputs. 5. System creates user account in Users DB. 6. If role is Supplier, account is flagged as Pending Verification. 7. User is redirected to Login page.
Alternative Flows	Email Already Exists system shows error and prompts different email or login. Weak Password user is prompted to correct it.
Business Rules	Email must be unique. Password must be encrypted using Bcrypt.js. Role must be selected at registration.
Assumptions	User has a stable internet connection.

UC-2: User Login

Use Case ID	UC-2
Use Case Name	User Login
Actors	Admin, Supplier, Wholesaler, Retailer, Customer
Description	Allows a registered user to log in using email and password. A JWT token is issued to manage role-based session access.
Trigger	User opens the platform and clicks Login.
Preconditions	User must already be registered. Supplier accounts must have Admin approval.
Postconditions	User is authenticated and redirected to their role-specific dashboard. JWT token is issued.
Normal Flow	1. User opens the platform. 2. User navigates to the Login page. 3. User enters email and password. 4. System validates credentials against Users DB. 5. System issues a JWT token. 6. User is redirected to their role-specific dashboard.
Alternative Flows	Invalid Credentials system shows error, user remains on login page. Pending Seller login denied with message to wait for Admin approval.
Business Rules	Email and password must match registered credentials. JWT token required for all protected routes. Token expires after a set duration.
Assumptions	User has a stable internet connection.

UC-3: Seller Verification by Admin

Use Case ID	UC-3
Use Case Name	Seller Verification by Admin

Actors	Admin, Seller
Description	Allows the Admin to review newly registered Seller accounts and approve or reject them. Only approved suppliers can list products.
Trigger	A new Seller registers. Admin opens the Supplier Verification section on the Admin dashboard.
Preconditions	A Seller account with Pending status exists in Users DB. Admin must be logged in.
Postconditions	Seller account status is updated to Approved or Rejected. Approved supplier can add product listings.
Normal Flow	1. Seller registers on the platform. 2. Account is flagged as Pending Verification.
	3. Admin logs in to the Admin dashboard. 4. Admin opens Supplier Verification section. 5. Admin reviews supplier details. 6. Admin clicks Approve. 7. System updates supplier status to Approved in Users DB. 8. Seller is notified via email.
Alternative Flows	Admin Rejects system updates status to Rejected. Seller is notified and cannot list products.
Business Rules	Only Admin can approve or reject supplier accounts. Suppliers without Approved status cannot add product listings.
Assumptions	Admin has access to the Admin dashboard. Email notification service is operational.

UC-4: Manage Product Listings

Use Case ID	UC-4
Use Case Name	Manage Product Listings
Actors	Supplier, Wholesaler, Retailer
Description	Allows a verified Seller to add new product listings and manage existing ones including editing details, updating stock and uploading images.

Trigger	Verified Seller logs in and navigates to Product Management on the Seller dashboard.
Preconditions	Seller must be logged in and have Admin-Approved status.
Postconditions	Product is added, updated or deleted in the Products DB. Stock level is updated accordingly.
Normal Flow	1. Seller logs in. 2. Seller opens Product Management dashboard. 3. Seller clicks Add Product. 4. Seller enters product name, description, category, price, min wholesale quantity and stock. 5. Seller uploads product image via Multer. 6. System validates the inputs. 7. Product is saved to Products DB.
Alternative Flows	Edit Product Seller selects an existing product and updates details. Delete Product Seller removes a product listing.
Business Rules	Only Admin-verified sellers can add or edit products. Stock level must not go below zero. Image uploads are subject to Multer file size restrictions.
Assumptions	Seller has correct product information and uses genuine product images.

UC-5: Browse and Search Products

Use Case ID	UC-5
Use Case Name	Browse and Search Products

Actors	Wholesaler, Retailer, Customer
Description	Allows authenticated buyers to browse the product catalog, search by name or category and view detailed product information before placing an order.

Trigger	Buyer logs in and navigates to the Marketplace or Product Listing page.
Preconditions	Buyer must be authenticated. Products from verified suppliers must be available in Products DB.
Postconditions	Buyer views product listing. Buyer can proceed to place an order from the product detail page.
Normal Flow	1. Buyer logs in. 2. Buyer navigates to the Marketplace. 3. System loads all verified supplier product listings. 4. Buyer enters a search term or selects a category filter. 5. System displays filtered product results. 6. Buyer clicks on a product card. 7. Product detail page is shown with name, description, price, stock and supplier.
Alternative Flows	No Results Found system displays 'No products found' message. Out of Stock product displayed with Stock Unavailable label.
Business Rules	Only products from Admin-verified suppliers are shown. Price comparison available for the same product from multiple suppliers.
Assumptions	Buyer has a stable internet connection.

UC-6: Place Order

Use Case ID	UC-6
Use Case Name	Place Order
Actors	Wholesaler, Retailer, Customer
Description	Allows authenticated buyers to place an order for selected products by specifying quantity and shipping details.

Trigger	Buyer clicks Place Order on the product detail page or cart page.
Preconditions	Buyer must be authenticated. Selected product must be in stock.
Postconditions	Order is created in Orders DB with status Created. Stock level is reserved. Supplier receives a new order notification.
Normal Flow	1. Buyer selects a product and clicks Add to Cart. 2. Buyer reviews cart and enters shipping details. 3. Buyer clicks Place Order. 4. System checks product stock availability. 5. System creates an order record in Orders DB with status Created. 6. System sends a new order email notification to the Supplier via Nodemailer.
Alternative Flows	Insufficient Stock system shows error and prevents order placement. Order Cancelled buyer or Admin cancels and status is updated to Cancelled.
Business Rules	Only authenticated users can place orders. Order quantity must not exceed available stock. Order status must follow the defined lifecycle.
Assumptions	Buyer has entered correct shipping details.

UC-7: Make Payment and Download Invoice

Use Case ID	UC-7
Use Case Name	Make Payment and Download Invoice
Actors	Wholesaler, Retailer, Customer, Stripe
Description	Allows the buyer to complete secure online payment through Stripe. On successful payment, a digital PDF invoice is generated and made available for download.
Trigger	Buyer clicks Proceed to Payment on the order summary page.
Preconditions	An order must exist in Orders DB with status Pending. Stripe payment gateway must be available.

Postconditions	Payment is processed and recorded in Payments DB. Order status updated to Paid. PDF invoice generated. Buyer receives invoice download link.
Normal Flow	1. Buyer clicks Proceed to Payment. 2. Stripe payment form is shown. 3. Buyer enters card details and submits. 4. System sends payment request to Stripe.
	5. Stripe confirms the transaction. 6. System records payment in Payments DB. 7. System updates order status to Paid. 8. System generates PDF invoice. 9. Buyer is shown the invoice download button.
Alternative Flows	Payment Failed Stripe returns failure. System shows error and order status remains Pending. Buyer can retry.
Business Rules	All payments must be processed through Stripe. Invoice generated only after Stripe payment confirmation. Online payments depend on third party gateway availability.
Assumptions	Stripe services are operational. Buyer has a valid payment method.

UC-8: Submit Bulk Order Request for Bidding

Use Case ID	UC-8
Use Case Name	Submit Bulk Order Request for Bidding
Actors	Wholesaler, Retailer
Description	Allows a buyer to submit a bulk order request specifying the product, required quantity and additional requirements. Verified suppliers can then view the request and submit bids.
Trigger	Buyer clicks Request Bulk Order on the product listing or bidding page.
Preconditions	Buyer must be authenticated. Product must exist in the Products DB.

Postconditions	Bulk order request is saved in Bids DB. Verified suppliers are notified via Nodemailer.
Normal Flow	1. Buyer selects a product. 2. Buyer clicks Request Bulk Order. 3. Buyer enters required quantity and any notes. 4. System saves the bulk order request in Bids DB with status Open. 5. System sends email notification to all verified suppliers via Nodemailer.
Alternative Flows	No Response from Suppliers buyer can cancel the request or resubmit.
Business Rules	Only authenticated buyers can submit bulk order requests. Notifications depend on Nodemailer availability.
Assumptions	Buyer has a stable internet connection.

UC-9: Submit and Manage Bids

Use Case ID	UC-9
Use Case Name	Submit and Manage Bids
Actors	Supplier, Wholesaler, Retailer
Description	Allows a verified Supplier to view open bulk order requests and submit a bid. The buyer can then view all bids and accept the best offer. The accepted bid initiates the order process.
Trigger	Supplier opens Bidding section on Supplier dashboard. Buyer opens My Bids section on their dashboard.
Preconditions	Supplier must be Admin-verified. A bulk order request must exist in Bids DB.
Postconditions	Bid is saved in Bids DB. If accepted, order process is initiated. Supplier is notified of acceptance.

Normal Flow	1. Supplier opens the Bidding section. 2. System loads open bulk order requests. 3. Supplier selects a request and submits a bid amount. 4. Bid is saved in Bids DB. 5. Buyer views all bids for their request. 6. Buyer selects and accepts the best bid. 7. System creates an order from the accepted bid. 8. System notifies the winning supplier via Nodemailer.
Alternative Flows	Supplier Cancels Bid supplier can cancel a submitted bid before it is accepted. No Bids Received buyer sees 'No Bids Yet' message.
Business Rules	Only Admin-verified suppliers can submit bids. A buyer can only accept one bid per request. Once accepted, a bid cannot be modified.
Assumptions	Email notification service is operational.

UC-10: Track Order Status

Use Case ID	UC-10
Use Case Name	Track Order Status
Actors	Wholesaler, Retailer, Customer
Description	Allows a buyer to view their order history and track the current status of each order through the full lifecycle: Created, Pending, Paid, Completed and Cancelled.
Trigger	Buyer clicks Order History on their dashboard.
Preconditions	Buyer must be authenticated. At least one order must exist in Orders DB.
Postconditions	Buyer views updated order status and can download invoice for paid orders.

Normal Flow	1. Buyer navigates to Order History on the dashboard. 2. System fetches all orders for the buyer from Orders DB. 3. Buyer views list of orders with status and date. 4. Buyer clicks on a specific order to see details. 5. If order is Paid, buyer clicks Download Invoice.
Alternative Flows	No Orders system displays 'No orders placed yet' message.
Business Rules	Invoice download is only available for orders with Paid status.
Assumptions	Buyer has placed at least one order.

UC-11: Admin User Management

Use Case ID	UC-11
Use Case Name	Admin User Management
Actors	Admin
Description	Allows the Admin to view, manage and delete user accounts across all roles from the Admin dashboard.
Trigger	Admin opens the User Management section of the Admin dashboard.
Preconditions	Admin must be authenticated.
Postconditions	User record is updated or deleted in the PostgreSQL Users DB.
Normal Flow	1. Admin logs in to the Admin dashboard. 2. Admin selects User Management. 3. System loads all registered users from Users DB. 4. Admin can view, edit or delete any user account. 5. System updates the Users DB accordingly.
Alternative Flows	Database Operation Failure → system displays an error message.
Business Rules	Sensitive actions such as deletion require confirmation. Only Admin can perform user management actions.
Assumptions	PostgreSQL database services are operational.

UC-12: Admin Product Management

Use Case ID	UC-12
Use Case Name	Admin Product Management
Actors	Admin
Description	Allows the Admin to view, edit and remove product listings from the platform and flag inappropriate or incorrect listings.
Trigger	Admin opens the Product Management section of the Admin dashboard.
Preconditions	Admin must be authenticated.
Postconditions	Product record is updated or removed from the Products DB.
Normal Flow	1. Admin opens Product Management. 2. System loads all product listings from Products DB. 3. Admin can view, edit or delete any product. 4. System updates the Products DB.
Alternative Flows	Database Operation Failure system shows an error message.
Business Rules	Sensitive actions require confirmation. Only Admin can remove or edit supplier product listings.
Assumptions	PostgreSQL database services are operational.

UC-13: Admin Order Management

Use Case ID	UC-13
Use Case Name	Admin Order Management
Actors	Admin

Description	Allows the Admin to view all orders across all users, monitor order statuses and intervene to cancel or update orders if needed.
Trigger	Admin opens the Order Management section of the Admin dashboard.
Preconditions	Admin must be authenticated.
Postconditions	Order status is updated in the Orders DB.
Normal Flow	1. Admin opens Order Management. 2. System loads all orders from Orders DB. 3. Admin views order details including buyer, supplier, status and amount. 4. Admin can update order status or cancel an order. 5. System updates the Orders DB.
Alternative Flows	Database Operation Failure system shows an error message.
Business Rules	Only Admin can force-cancel or modify orders outside the normal buyer initiated flow.
Assumptions	PostgreSQL database services are operational.

UC-14: Generate System Reports

Use Case ID	UC-14
Use Case Name	Generate System Reports
Actors	Admin
Description	Allows the Admin to generate reports and view analytics about platform activity including total orders, revenue, user registrations and supplier performance.
Trigger	Admin opens the Reports section of the Admin dashboard.
Preconditions	Admin must be authenticated. Data must exist in the PostgreSQL databases.
Postconditions	Reports and analytics are displayed on the Admin dashboard.

Normal Flow	1. Admin opens the Reports section. 2. System queries data from relevant PostgreSQL databases. 3. System displays summary reports including orders, revenue and user activity. 4. Admin can view detailed breakdowns per time period.
Alternative Flows	No Data Available system shows 'No data to display' message.
Business Rules	Reports are generated in real time from the database. Only Admin has access to the Reports section.
Assumptions	PostgreSQL database services are operational. Sufficient data exists in the system.

Appendix B: Coding Standards

General Coding Standards & Guidelines

1.Consistent Variable Naming

We use camel case for variables and pascal case for classes.

2. Meaningful Naming

Variables and fields are named clearly to describe their purpose.

3.Immutable Fields

Fields are kept constant after initialization using const or read only where required.

4.Controlled Access

Sensitive data is accessed using get only properties to prevent unauthorized updates.

5.Function Naming

Functions are named based on their functionality.

6.Comments Usage

Comments are added for feedback and for user while inappropriate comments can be removed or hide by admin.

7.Indentation & Formatting

Proper indentation is used to clearly define loops, conditions, and blocks for better readability.

8.File & Folder Naming

Files and folders are named according to their content.

9.Class Structure

Each class follows a clear structure with properties, constructors and methods

10. Code Organization

Web-App is divided into frontend, backend and database. Each class is stored in a separate file.

11.Minimum Scope Usage

Variables are declared only where needed to avoid unnecessary access.

12.Access Modifiers

Proper use of public and private is followed to secure data and restrict access.

13.High Cohesion & Low Coupling

Each module performs a specific task and is connected to others.

14.DRY Principle

- Reusable functions are created for repeated tasks.
- Matching logic is written once and reused.
- Methods are kept short and simple.
- Avoid long and complex functions.

15.Single Responsibility Principle

Each class handles only one responsibility

16.Extenable Code

Classes and functions are written in a way that they can be extended without modifying existing code.

17.Exception Handling

Errors are handled using try-catch blocks.

18.Logging

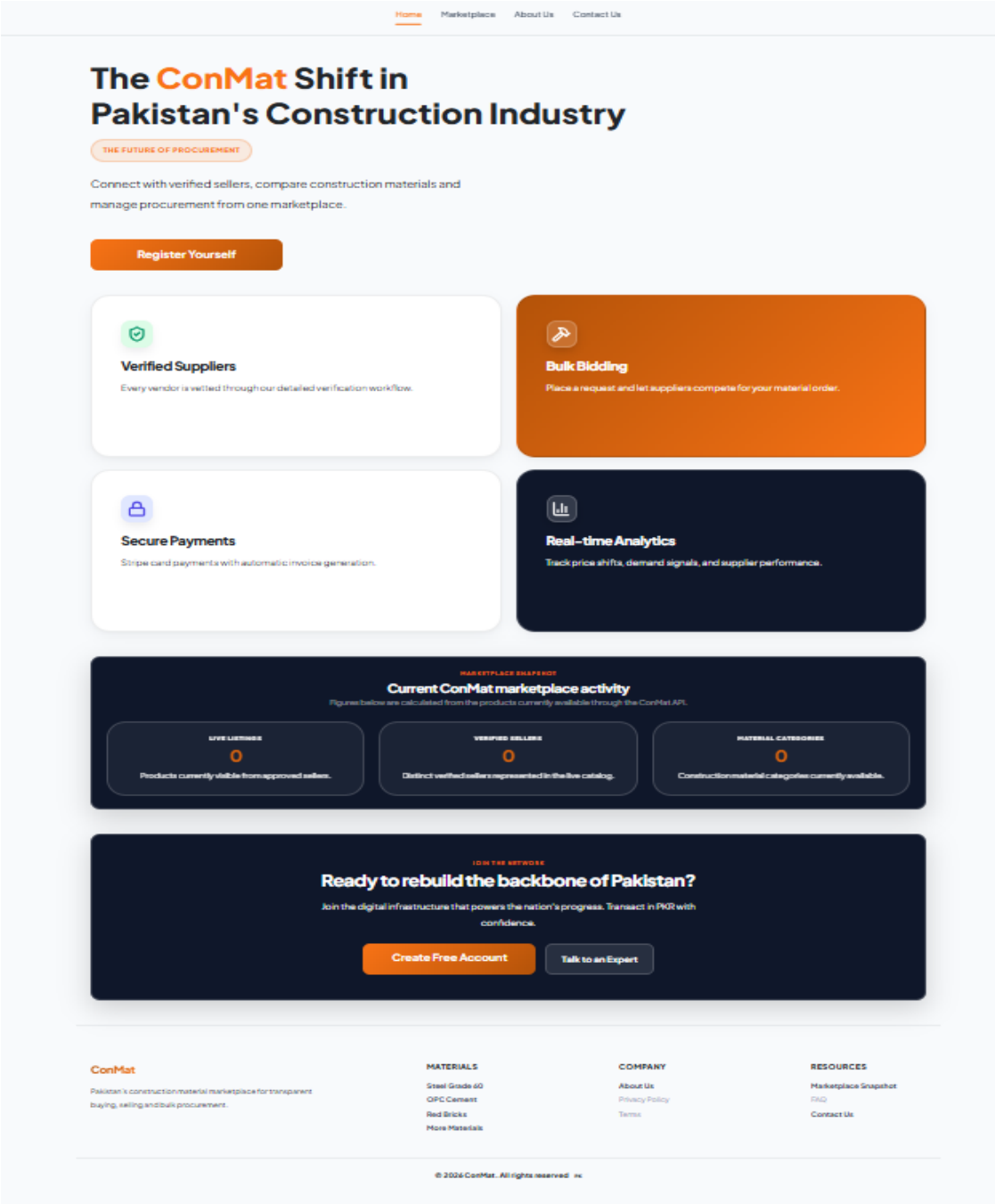
Important events like report submission, verification and errors are logged for tracking.

19.Consistent Logging Style

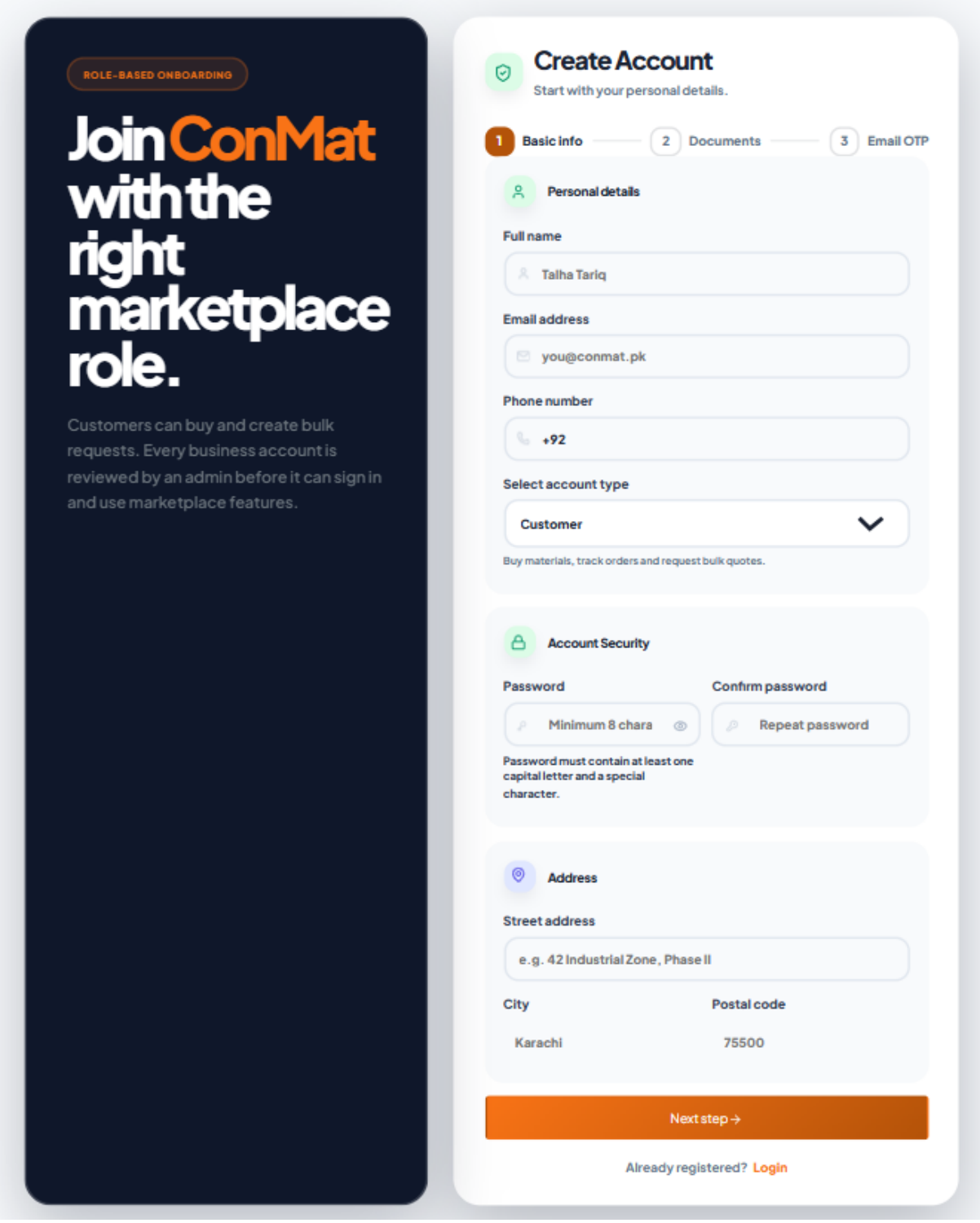
A uniform logging format is used across the system.

Appendix C: Prototypes

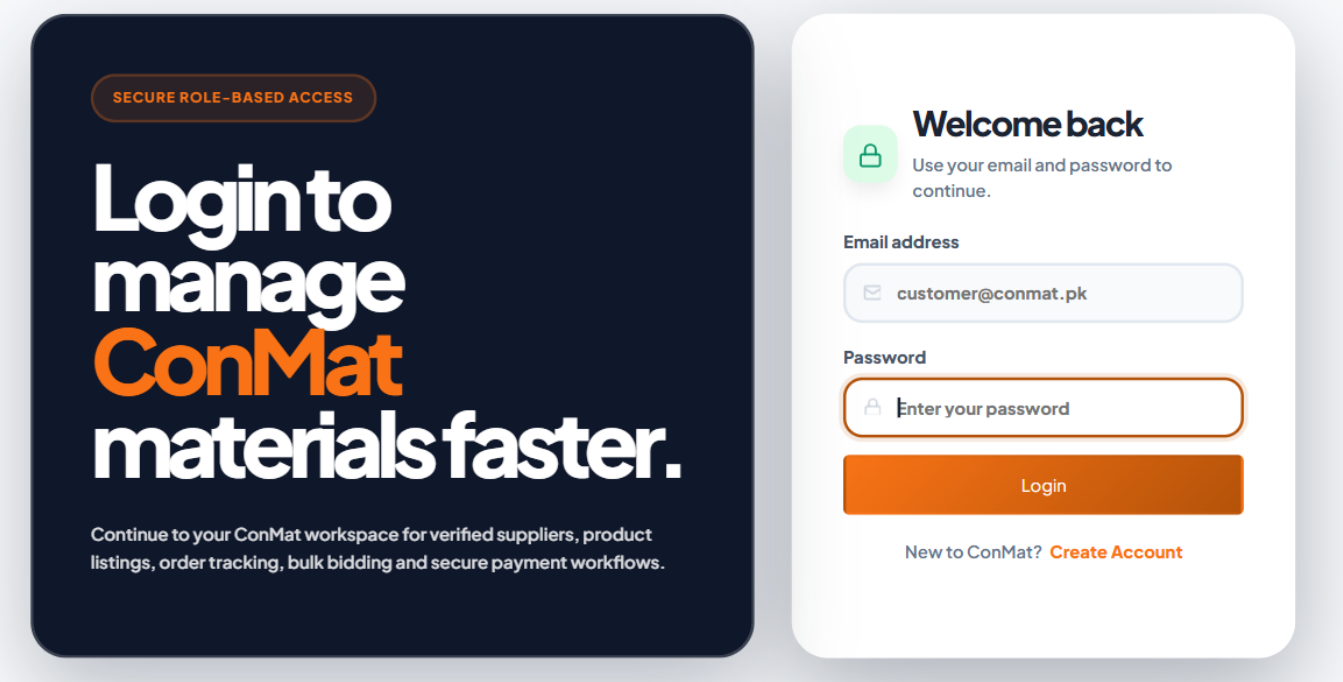
Landing Page:



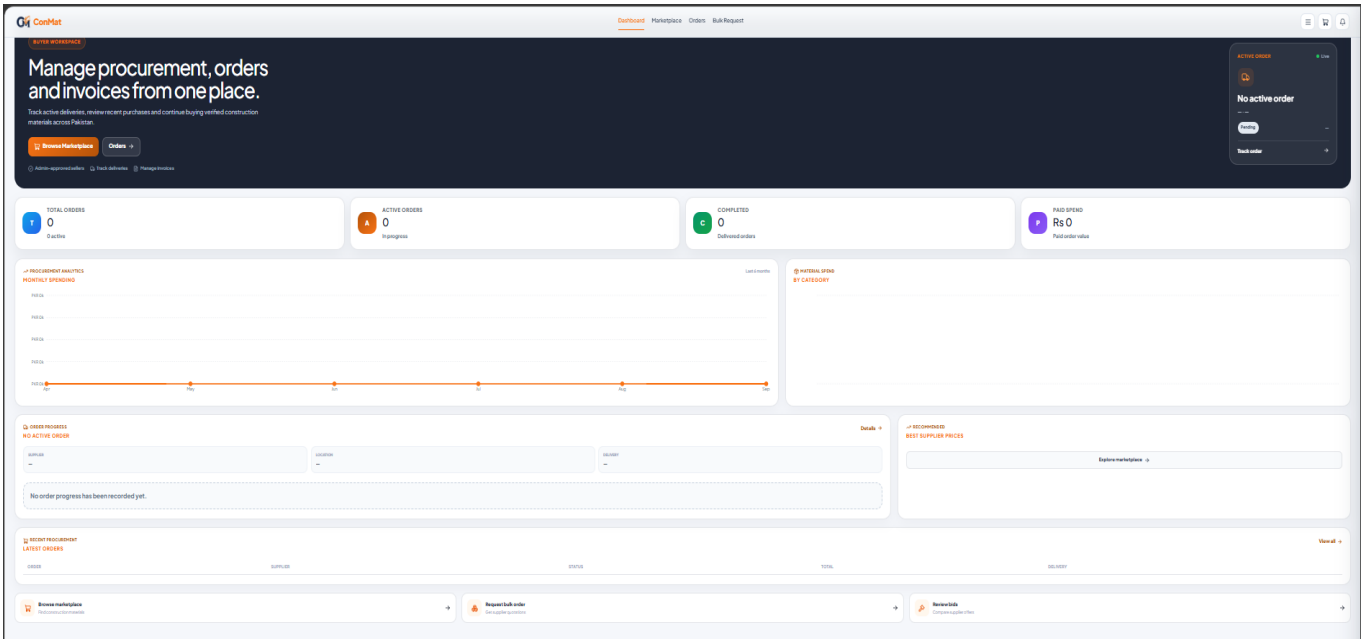
Registration Page:



Login Page:



Customer Dashboard:



Marketplace Dashboard:

